

BÀI 2: ĐỊNH DẠNG VÀ BỐ CỤC VỚI CSS, THIẾT KẾ WEB RESPONSIVE

1. Mục tiêu của bài:

- Hiểu được vai trò của CSS trong việc định dạng và bố cục trang web.
- Sử dụng được các thuộc tính CSS cơ bản để định dạng văn bản, màu sắc, kích thước, khoảng cách và đường viền.
- Thiết kế được bố cục trang web hiện đại bằng Flexbox hoặc Grid, đảm bảo tương thích trên nhiều thiết bị (responsive).

2. Nội dung bài:

2.1. Giới thiệu CSS và cách áp dụng

CSS là ngôn ngữ dùng để mô tả cách hiển thị của tài liệu HTML. Nếu HTML giữ vai trò tổ chức nội dung và xác định cấu trúc trang, thì CSS đảm nhận nhiệm vụ trình bày như màu sắc, phông chữ, khoảng cách, đường viền, bố cục và khả năng đáp ứng trên các kích thước màn hình khác nhau. Khi học phát triển web, chúng ta cần nhìn CSS không chỉ như tập hợp các dòng lệnh trang trí, mà như một cơ chế điều khiển giao diện theo quy tắc. Một phần tử HTML có thể chịu tác động đồng thời từ nhiều quy tắc CSS, và trình duyệt phải tính toán xem quy tắc nào được áp dụng sau cùng. Vì vậy, ngay từ đầu, chúng ta cần nắm chắc ba nền tảng là cú pháp CSS, cách gắn CSS vào HTML, và nguyên tắc cascade cùng kế thừa.

Trong thực tế phát triển giao diện, cùng một trang web có thể sử dụng hàng trăm hoặc hàng nghìn quy tắc CSS. Nếu không hiểu nguyên lý hoạt động, người học thường gặp lỗi như đổi màu không có tác dụng, chỉnh kích thước nhưng giao diện không thay đổi, hoặc viết thêm CSS càng nhiều càng rối. Do đó, phần này giúp chúng ta thiết lập nền móng để học các nội dung sâu hơn như box model, flexbox, grid, responsive design và animation.

2.1.1. Khái niệm selector, thuộc tính, giá trị trong CSS.

Một quy tắc CSS cơ bản gồm hai phần lớn là selector và declaration block. Selector dùng để xác định phần tử HTML nào sẽ bị tác động. Declaration block là khối khai báo nằm trong cặp dấu ngoặc nhọn, bên trong chứa một hoặc nhiều cặp thuộc tính và giá trị.

Cú pháp tổng quát:

```
selector {  
  property: value;  
}
```

Ví dụ:

```
h1 {  
  color: blue;  
  font-size: 32px;  
}
```

Trong ví dụ này, h1 là selector, color và font-size là thuộc tính, còn blue và 32px là giá trị tương ứng. Trình duyệt hiểu rằng mọi phần tử <h1> trên trang sẽ có màu xanh và cỡ chữ 32 pixel, trừ khi bị quy tắc khác có độ ưu tiên cao hơn ghi đè.

a. Selector

Selector là thành phần đầu tiên của quy tắc CSS, dùng để chọn đối tượng cần định dạng. Đây là khâu rất quan trọng, vì chọn sai selector thì phần CSS vẫn đúng cú pháp nhưng không tác động vào phần tử mong muốn.

Một số selector cơ bản mà chúng ta thường dùng ở giai đoạn đầu gồm:

Loại selector	Cú pháp	Ý nghĩa	Ví dụ HTML áp dụng
Selector theo tên thẻ	p	Chọn tất cả phần tử cùng tên thẻ	...
Selector theo id	#menu	Chọn phần tử có id cụ thể	<div id="menu">...</div>
Selector theo class	.card	Chọn tất cả phần tử có class tương ứng	<div class="card">...</div>
Selector kết hợp	div p	Chọn phần tử con theo quan hệ cấu trúc	...
Selector nhiều phần tử	h1, h2, h3	Áp dụng cùng một kiểu cho nhiều selector	<h1>, <h2>, <h3>

Bảng 3.1: Các loại selector cơ bản trong CSS

Ví dụ:

```
p {  
  color: #333;  
}  
#main-title {  
  text-align: center;  
}  
.note {
```

```
background-color: #fff3cd;
}
```

Trong đó:

- p tác động lên tất cả đoạn văn.
- #main-title chỉ tác động lên một phần tử có id="main-title".
- .note tác động lên mọi phần tử mang class note.

b. Thuộc tính

Thuộc tính là đặc trưng giao diện mà chúng ta muốn điều chỉnh. CSS có rất nhiều thuộc tính, nhưng có thể phân nhóm thành các loại tiêu biểu như:

Nhóm thuộc tính	Ví dụ
Màu sắc và nền	color, background-color, background-image
Văn bản và phong chữ	font-size, font-family, font-weight, text-align
Kích thước	width, height, max-width
Khoảng cách	margin, padding
Viền	border, border-radius
Hiển thị và bố cục	display, position, flex, grid

Bảng 3.2: Nhóm thuộc tính CSS thường gặp

Ví dụ:

```
.box {
  width: 300px;
  padding: 20px;
  border: 1px solid #ccc;
}
```

Ở đây, width, padding, border là các thuộc tính.

c. Giá trị

Giá trị là nội dung gán cho thuộc tính. Tùy từng thuộc tính, kiểu giá trị có thể là màu, số đo, từ khóa hoặc danh sách giá trị.

Ví dụ:

```
color: red;
font-size: 18px;
text-align: center;
width: 50%;
```

Trong các dòng trên:

- red là giá trị màu.
- 18px là giá trị độ dài.
- center là từ khóa căn giữa.
- 50% là giá trị phần trăm.

Chúng ta cần chú ý rằng không phải mọi thuộc tính đều nhận cùng loại giá trị. Chẳng hạn color không thể nhận giá trị 20px, còn width không thể nhận giá trị bold. Do đó, ngoài việc nhớ cú pháp, chúng ta cần hiểu quan hệ đúng giữa thuộc tính và kiểu giá trị.

d. Phân tích một quy tắc CSS hoàn chỉnh

Xét ví dụ:

```
.article-title {  
  color: #1d4ed8;  
  font-size: 28px;  
  margin-bottom: 12px;  
}
```

Phân tích:

- .article-title là selector theo class, chọn các phần tử có class article-title.
- color: #1d4ed8; đặt màu chữ xanh.
- font-size: 28px; quy định cỡ chữ.
- margin-bottom: 12px; tạo khoảng cách phía dưới phần tử.

Qua đó, chúng ta thấy một selector có thể đi kèm nhiều cặp thuộc tính và giá trị để mô tả đầy đủ cách hiển thị của phần tử.

2.1.2. Ba cách áp dụng CSS: nội tuyến, nội bộ và tách riêng tệp CSS

CSS có thể được áp dụng vào HTML theo ba cách phổ biến. Mỗi cách đều hợp lệ về mặt kỹ thuật, nhưng mục đích sử dụng và mức độ phù hợp trong dự án là khác nhau.

a. CSS nội tuyến

CSS nội tuyến là cách viết trực tiếp trong thuộc tính style của phần tử HTML.

Ví dụ:

```
<p style="color: red; font-size: 18px;">Đây là đoạn văn màu  
đỏ.</p>
```

Trong cách này, toàn bộ quy tắc CSS gắn chặt với chính phần tử đó. Trình duyệt không cần tra cứu nơi khác mà áp dụng ngay nội dung trong style.

Đặc điểm kỹ thuật

- Tác động trực tiếp lên một phần tử.
- Có độ ưu tiên cao hơn CSS nội bộ và CSS tệp ngoài trong nhiều trường hợp thông thường.
- Khó tái sử dụng.
- Gây rối mã HTML khi số lượng thuộc tính nhiều.

Trường hợp sử dụng phù hợp

- Thử nhanh một định dạng nhỏ.
- Minh họa ngắn trong quá trình học.
- Xử lý tình huống đặc biệt cho một phần tử duy nhất.

Hạn chế

Khi xây dựng giao diện thực tế, nếu lạm dụng nội tuyến, mã HTML sẽ dài, khó đọc, khó bảo trì, và làm mất nguyên tắc tách biệt cấu trúc với trình bày.

b. CSS nội bộ

CSS nội bộ là cách đặt các quy tắc CSS trong thẻ `<style>` ở phần `<head>` của tài liệu HTML. Ví dụ:

```
<!DOCTYPE html>
<html lang="vi">
<head>
  <meta charset="UTF-8">
  <title>Ví dụ CSS nội bộ</title>
  <style>
    h1 {
      color: blue;
      text-align: center;
    }

    p {
      font-size: 18px;
      color: #444;
    }
  </style>
```

```
</head>
<body>
  <h1>Tiêu đề bài học</h1>
  <p>Nội dung đoạn văn bản.</p>
</body>
</html>
```

Đặc điểm kỹ thuật

- CSS và HTML vẫn nằm trong cùng một tệp.
- Dễ quản lý hơn CSS nội tuyến.
- Có thể áp dụng cho nhiều phần tử trong cùng một trang.
- Không thuận tiện khi website có nhiều trang.

Trường hợp sử dụng phù hợp

- Trang web nhỏ chỉ có một tệp HTML.
- Bài thực hành ban đầu.
- Ví dụ minh họa trên lớp hoặc trên CodePen.

Hạn chế

Nếu website có nhiều trang, việc lặp lại cùng một đoạn CSS trong từng tệp HTML sẽ gây dư thừa và khó đồng bộ.

c. CSS tách riêng tệp

Đây là cách chuyên nghiệp và được sử dụng phổ biến nhất. CSS được đặt trong một tệp riêng, ví dụ style.css, sau đó liên kết với HTML thông qua thẻ <link>.

Ví dụ HTML:

```
<!DOCTYPE html>
<html lang="vi">
<head>
  <meta charset="UTF-8">
  <title>Ví dụ CSS tệp ngoài</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1>Học CSS cơ bản</h1>
  <p class="intro">Đây là đoạn giới thiệu.</p>
</body>
</html>
```

Ví dụ style.css:

```
h1 {  
    color: green;  
    text-align: center;  
}  
.intro {  
    font-size: 18px;  
    color: #333;  
}
```

Đặc điểm kỹ thuật

- Tách biệt rõ giữa cấu trúc HTML và trình bày CSS.
- Có thể dùng chung một tệp CSS cho nhiều trang HTML.
- Thuận lợi cho bảo trì, mở rộng và tái sử dụng.
- Phù hợp với dự án thật.

Trường hợp sử dụng phù hợp

- Website nhiều trang.
- Dự án học tập có tổ chức thư mục.
- Bài tập cần dễ sửa, dễ kiểm tra và dễ mở rộng.
- Hạn chế
- Cần liên kết đúng đường dẫn tệp.
- Nếu liên kết sai, trang HTML vẫn chạy nhưng CSS không áp dụng.

d. Bảng so sánh ba cách áp dụng CSS

Tiêu chí	Nội tuyến	Nội bộ	Tệp CSS riêng
Vị trí viết CSS	Trong thuộc tính style của thẻ HTML	Trong thẻ <style>	Trong tệp .css riêng
Phạm vi áp dụng	Một phần tử	Một trang	Nhiều trang
Mức độ tái sử dụng	Rất thấp	Trung bình	Cao
Khả năng bảo trì	Kém	Khá	Tốt
Phù hợp dạy học ban đầu	Có	Có	Rất nên áp dụng
Phù hợp dự án thực tế	Hạn chế	Hạn chế	Tốt nhất

Bảng 3.3: So sánh ba cách áp dụng CSS

Từ phân tích trên, trong học tập ban đầu chúng ta có thể dùng cả ba cách để hiểu cơ chế hoạt động. Tuy nhiên, khi tổ chức mã nguồn nghiêm túc, chúng ta nên ưu tiên CSS tệp riêng để đảm bảo tính rõ ràng và khả năng mở rộng.

2.1.3. Thứ tự ưu tiên (cascade) và kế thừa trong CSS.

Một trong những nội dung quan trọng nhất của CSS là hiểu vì sao nhiều quy tắc cùng tác động lên một phần tử nhưng chỉ một số giá trị được áp dụng sau cùng. Cơ chế đó gọi là cascade. Song song với đó, có những thuộc tính không cần viết trực tiếp trên phần tử con mà vẫn được nhận từ phần tử cha, cơ chế đó gọi là kế thừa.

a. Cascade là gì

Cascade là quy tắc xác định giá trị cuối cùng khi nhiều khai báo CSS cùng áp dụng lên một phần tử. Trình duyệt không chọn ngẫu nhiên mà dựa trên hệ thống ưu tiên gồm:

- Mức quan trọng như !important
- Độ ưu tiên của selector
- Thứ tự xuất hiện trong mã nguồn

Ví dụ:

```
<p class="text" id="special">Nội dung minh họa</p>
p {
  color: black;
}
.text {
  color: blue;
}
#special {
  color: red;
}
```

Ở đây, phần tử <p> đồng thời khớp với ba selector là p, .text, và #special. Kết quả cuối cùng chữ sẽ có màu đỏ, vì selector id có độ ưu tiên cao hơn selector class và selector tên thẻ.

b. Độ ưu tiên của selector

Mỗi loại selector có mức ưu tiên khác nhau. Mức ưu tiên có thể hiểu như “trọng số” khi trình duyệt so sánh các quy tắc. Thứ tự thường gặp từ thấp đến cao:

Mức ưu tiên	Loại khai báo
Thấp	Selector tên thẻ

Trung bình	Selector class, thuộc tính, pseudo-class
Cao	Selector id
Rất cao	CSS nội tuyến
Đặc biệt	!important

Bảng 3.4: Mức ưu tiên thường gặp của selector và cách khai báo

Ví dụ:

```
<h2 id="title" class="heading" style="color: orange;">Tiêu
đề</h2>
h2 {
  color: blue;
}
.heading {
  color: green;
}
#title {
  color: red;
}
```

Kết quả cuối cùng là màu cam vì CSS nội tuyến có độ ưu tiên cao hơn các selector thông thường.

c. Thứ tự xuất hiện trong mã nguồn

Khi hai quy tắc có cùng độ ưu tiên, quy tắc viết sau sẽ được áp dụng. Ví dụ:

```
p {
  color: blue;
}
p {
  color: green;
}
```

Kết quả cuối cùng là màu xanh lá, vì quy tắc thứ hai xuất hiện sau. Điều này giúp chúng ta hiểu vì sao khi sửa CSS, đôi khi phần khai báo cũ không bị lỗi nhưng vẫn không còn tác dụng vì đã bị một khai báo khác cùng mức ưu tiên nhưng nằm sau ghi đè.

d. Vai trò của !important

Khai báo !important làm tăng mức ưu tiên của một thuộc tính lên rất cao. Ví dụ:

```
p {
  color: blue !important;
}
#note {
  color: red;
}
```

Nếu một phần tử `<p id="note">...</p>` nhận cả hai quy tắc trên, màu cuối cùng vẫn là xanh, vì `!important` được ưu tiên hơn quy tắc thông thường.

Trong thực hành, chúng ta cần hiểu `!important` nhưng không nên lạm dụng. Khi dùng quá nhiều, mã CSS trở nên khó kiểm soát, làm mất tác dụng của hệ thống cascade tự nhiên.

e. Kế thừa trong CSS

Kế thừa là cơ chế một số thuộc tính của phần tử cha tự động truyền xuống phần tử con nếu phần tử con chưa khai báo giá trị riêng. Ví dụ:

```
<div class="container">
  <p>Đây là đoạn văn nằm trong khối container.</p>
</div>
.container {
  color: navy;
  font-family: Arial, sans-serif;
}
```

Kết quả là đoạn `<p>` bên trong cũng hiển thị màu xanh đậm và dùng phông Arial, dù chúng ta không viết CSS trực tiếp cho `p`. Các thuộc tính thường có tính kế thừa gồm:

Thuộc tính hay kế thừa	Ý nghĩa
color	Màu chữ
font-family	Phông chữ
font-size	Cỡ chữ
line-height	Chiều cao dòng
text-align	Căn lề văn bản

Bảng 3.5: Thuộc tính CSS thường có tính kế thừa

Ngược lại, nhiều thuộc tính bố cục không tự kế thừa, ví dụ:

Thuộc tính không tự kế thừa	Ý nghĩa
margin	Khoảng cách ngoài

padding	Khoảng cách trong
border	Viền
width	Chiều rộng
background-color	Màu nền

Bảng 3.6: Thuộc tính CSS thường không tự kế thừa

Ví dụ:

```
.container {
  background-color: yellow;
}
```

Phần tử con không tự động nhận màu nền vàng như phần tử cha theo kiểu kế thừa. Màu nền chỉ áp dụng trên chính phần tử .container.

f. Phân biệt cascade và kế thừa

Hai cơ chế này thường dễ bị nhầm lẫn, nên chúng ta cần tách bạch:

Nội dung	Cascade	Kế thừa
Bản chất	So sánh nhiều quy tắc cùng áp dụng lên một phần tử	Truyền một số thuộc tính từ cha xuống con
Mục tiêu	Chọn giá trị thắng cuối cùng	Giảm việc lặp lại khai báo
Phụ thuộc selector	Có	Không trực tiếp
Ví dụ	#id thắng .class	color của cha áp dụng cho con

Bảng 3.7: Phân biệt cascade và kế thừa trong CSS

Quy trình thực hành trên VS Code

Chúng ta sẽ tạo một trang HTML đơn giản để quan sát:

- selector, thuộc tính, giá trị
- ba cách áp dụng CSS
- thứ tự ưu tiên và kế thừa

Bước 1. Tạo cấu trúc thư mục:

```
css-basic/
├── index.html
└── style.css
```

Bước 2. Soạn tệp index.html

```
<!DOCTYPE html>
<html lang="vi">
<head>
```

```
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-
scale=1.0">
<title>Thực hành CSS cơ bản</title>
<style>
  p {
    color: blue;
  }
  .internal-text {
    font-size: 20px;
    font-style: italic;
  }
  #priority-demo {
    color: purple;
  }
  .parent-box {
    color: darkgreen;
    font-family: Arial, sans-serif;
    border: 2px solid #999;
    padding: 12px;
    margin-top: 20px;
  }
</style>
<link rel="stylesheet" href="style.css">
</head>
<body>
  <h1 id="main-title" style="color: orange;">Giới thiệu CSS</h1>
  <p>Đây là đoạn văn được chọn bằng selector tên thẻ.</p>
  <p class="internal-text">
    Đây là đoạn văn có class để minh họa CSS nội bộ.
  </p>
  <p id="priority-demo" class="highlight" style="color: red;">
    Đây là đoạn văn minh họa thứ tự ưu tiên của CSS.
  </p>
  <div class="parent-box">
```

```
    Đây là phần tử cha.  
    <p>Đây là phần tử con kế thừa màu chữ và phông chữ từ  
cha.</p>  
  </div>  
</body>  
</html>
```

Bước 3. Soạn tệp style.css

```
h1 {  
  text-align: center;  
  font-size: 32px;  
}  
.highlight {  
  color: green;  
  background-color: #f3f4f6;  
  padding: 8px;  
}  
.parent-box p {  
  border: 1px dashed #666;  
  padding: 8px;  
}
```

Bước 4. Mở tệp trong trình duyệt. Trong VS Code, chúng ta có thể:

- mở thư mục dự án
- nhấn chuột phải vào index.html
- chọn mở bằng trình duyệt hoặc dùng tiện ích Live Server nếu đã cài

Bước 5. Quan sát và phân tích kết quả

Đoạn:

```
p {  
  color: blue;  
}
```

tác động lên tất cả các thẻ <p>, nên những đoạn văn chưa bị quy tắc ưu tiên cao hơn ghi đè sẽ có màu xanh.

Đoạn:

```
.internal-text {  
  font-size: 20px;
```

```
font-style: italic;
}
```

áp dụng cho phần tử có class="internal-text", giúp chúng ta phân biệt giữa selector tên thẻ và selector class.

Phần tử:

```
<p id="priority-demo" class="highlight" style="color: red;">
```

chịu tác động của nhiều quy tắc:

- p { color: blue; }
- .highlight { color: green; }
- #priority-demo { color: purple; }
- style="color: red;"

Kết quả cuối cùng là màu đỏ vì CSS nội tuyến có độ ưu tiên cao nhất trong nhóm này.

Khởi:

```
.parent-box {
  color: darkgreen;
  font-family: Arial, sans-serif;
}
```

làm phần tử con <p> bên trong thừa hưởng màu chữ và phong chữ. Tuy nhiên đường viền và padding của phần tử con không phải do kế thừa từ .parent-box, mà do quy tắc riêng:

```
.parent-box p {
  border: 1px dashed #666;
  padding: 8px;
}
```

Qua đó chúng ta thấy rõ thuộc tính văn bản thường kế thừa, còn thuộc tính bố cục thì thường không.

Bảng sau đây sẽ tóm lược một số lỗi thường gặp khi mới học CSS

Lỗi	Nguyên nhân kỹ thuật	Cách xử lý
CSS không áp dụng	Sai selector hoặc sai đường dẫn tệp CSS	Kiểm tra tên class, id, tên tệp, vị trí tệp
Viết thiếu dấu ; hoặc { }	Lỗi cú pháp khiến trình duyệt bỏ qua khai báo	Soát lại từng dòng CSS

Đổi màu không thấy tác dụng	Bị quy tắc khác ưu tiên cao hơn ghi đè	Kiểm tra cascade, inline style, id, !important
Thuộc tính không kế thừa như mong đợi	Thuộc tính đó không thuộc nhóm kế thừa	Viết quy tắc trực tiếp cho phần tử con
Nhầm . với #	. dùng cho class, # dùng cho id	Kiểm tra lại HTML và CSS đồng thời

Bảng 3.8: Một số lỗi thường gặp khi học CSS

2.2. Định dạng văn bản và hộp (box model)

Trong thiết kế giao diện web, hai nhóm kiến thức nền tảng cần nắm rất chắc là định dạng văn bản và mô hình hộp. Văn bản là thành phần xuất hiện ở hầu hết mọi khu vực giao diện như tiêu đề, đoạn mô tả, nút bấm, liên kết, chú thích và thông báo. Mô hình hộp là cách trình duyệt tính kích thước thực tế, khoảng cách và phạm vi hiển thị của từng phần tử HTML.

Phần này giúp chúng ta tìm hiểu các thuộc tính CSS cơ bản để định dạng chữ và đoạn văn. Sau đó, chúng ta phân tích cấu trúc box model gồm content, padding, border và margin. Cuối cùng, chúng ta áp dụng kiến thức vào việc định dạng khối nội dung và tạo khoảng cách hợp lý giữa các phần tử trên trang.

2.2.1. Thuộc tính *font*, *color*, *text-align*, *text-decoration*, *line-height*,...

Các thuộc tính định dạng văn bản là nhóm thuộc tính điều khiển hình thức hiển thị của nội dung chữ. Trong quá trình thiết kế giao diện, chúng ta không chỉ làm cho văn bản “đẹp hơn”, mà còn phải làm cho nó dễ đọc, có thứ bậc rõ ràng, và phù hợp với mục đích truyền đạt thông tin.

a. Thuộc tính *font-family*

font-family dùng để chỉ định kiểu phong chữ. Một khai báo thường nên có nhiều lựa chọn dự phòng, để nếu trình duyệt không có phong đầu tiên thì có thể dùng phong sau. Ví dụ:

```
body {
  font-family: Arial, Helvetica, sans-serif;
}
```

Phân tích:

- Arial là lựa chọn ưu tiên.
- Helvetica là lựa chọn dự phòng.
- sans-serif là họ phong tổng quát nếu hai phong trước không có.

Việc khai báo theo chuỗi như vậy giúp trình duyệt hiển thị ổn định trên nhiều máy khác nhau.

b. Thuộc tính font-size

font-size quy định kích thước chữ. Đây là thuộc tính ảnh hưởng trực tiếp đến khả năng đọc. Ví dụ:

```
p {  
  font-size: 18px;  
}
```

Một số đơn vị thường gặp:

Đơn vị	Ý nghĩa	Ghi chú
px	pixel	Dễ hiểu, phổ biến cho người mới học
em	theo kích thước phần tử cha	Linh hoạt, mang tính tương đối
rem	theo kích thước gốc của trang	Thuận tiện cho thiết kế đồng bộ
%	phần trăm	Thường dùng trong một số tình huống đặc biệt

Bảng 3.9: Một số đơn vị kích thước thường dùng trong font-size

Trong giai đoạn đầu, chúng ta thường dùng px để dễ quan sát. Khi học sâu hơn về responsive design, rem sẽ rất quan trọng.

c. Thuộc tính font-weight

font-weight điều khiển độ đậm của chữ. Ví dụ:

```
h1 {  
  font-weight: bold;  
}  
strong {  
  font-weight: 700;  
}
```

Các giá trị thường dùng:

- normal: chữ thường
- bold: chữ đậm
- 100 đến 900: mức đậm từ mảnh đến rất đậm

Trong thiết kế nội dung, chúng ta dùng font-weight để làm nổi bật tiêu đề, nhấn mạnh thông tin quan trọng hoặc phân cấp thị giác giữa các vùng văn bản.

d. Thuộc tính color

color quy định màu chữ. Ví dụ:

```
p {  
  color: #333333;  
}
```

Trong thực tế, mã hex và RGB được dùng nhiều vì cho phép kiểm soát màu chính xác hơn.

e. Thuộc tính text-align

text-align dùng để căn lề ngang cho nội dung văn bản. Ví dụ:

```
h1 {  
  text-align: center;  
}  
p {  
  text-align: justify;  
}
```

Một số giá trị cho text-align:

Giá trị	Ý nghĩa
left	căn trái
center	căn giữa
right	căn phải
justify	căn đều hai bên

Bảng 3.10: Các giá trị cơ bản của text-align

Trong tài liệu học tập hoặc bài viết dài, justify giúp đoạn văn nhìn đều hơn, nhưng cần dùng thận trọng vì có thể tạo khoảng trắng không đều trên màn hình nhỏ.

f. Thuộc tính text-decoration

text-decoration điều khiển kiểu trang trí chữ, điển hình là gạch chân, gạch ngang hoặc bỏ gạch chân của liên kết. Ví dụ:

```
a {  
  text-decoration: none;  
}  
h2 {
```

```
text-decoration: underline;
}
```

Các giá trị cơ bản của text-decoration

Giá trị	Ý nghĩa
none	bỏ trang trí
underline	gạch chân
overline	gạch phía trên
line-through	gạch ngang giữa

Bảng 3.11: Các giá trị thường dùng của text-decoration

Trong phát triển web, text-decoration: none; thường được dùng để bỏ gạch chân mặc định của thẻ liên kết, sau đó kết hợp màu sắc hoặc hiệu ứng khác để tạo kiểu hiện đại hơn.

g. Thuộc tính line-height

line-height điều khiển chiều cao dòng, tức khoảng cách theo phương dọc giữa các dòng chữ. Đây là thuộc tính rất quan trọng để tăng khả năng đọc. Ví dụ:

```
p {
  line-height: 1.6;
}
```

Nếu line-height quá nhỏ, các dòng sát nhau và khó đọc. Nếu quá lớn, đoạn văn bị loãng và mất liên kết thị giác. Đối với nội dung văn bản thông thường, các giá trị từ 1.4 đến 1.8 thường cho cảm giác dễ đọc.

h. Một số thuộc tính văn bản thường dùng khác

Các giá trị thường dùng:

Thuộc tính	Ý nghĩa	Ví dụ
font-style	kiểu chữ	italic
text-transform	chuyển đổi chữ hoa, chữ thường	uppercase
letter-spacing	khoảng cách giữa các ký tự	1px
word-spacing	khoảng cách giữa các từ	4px
text-indent	thụt đầu dòng	20px

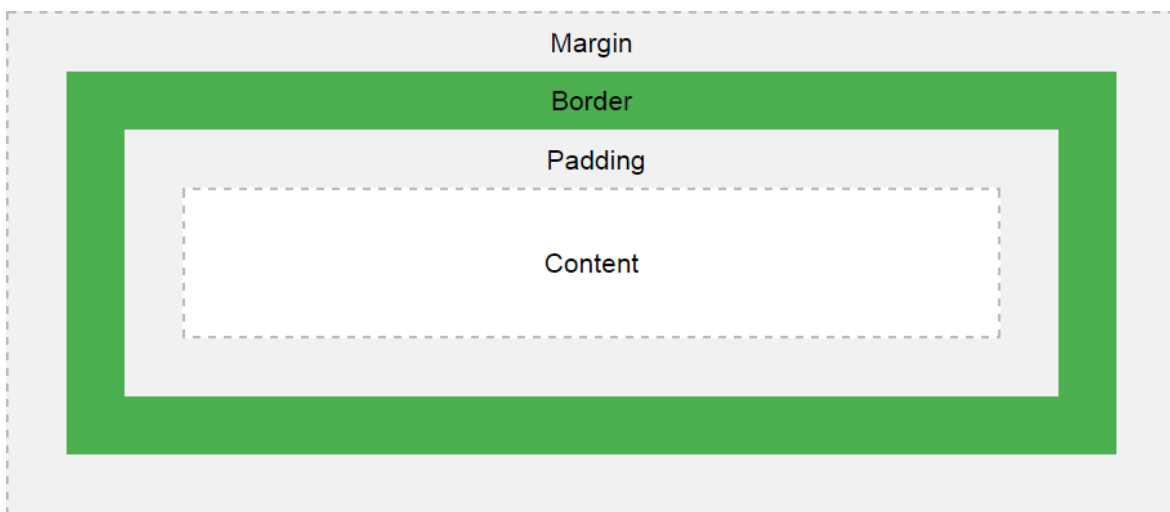
Bảng 3.12: Một số thuộc tính văn bản thường dùng khác

Ví dụ:

```
.subtitle {  
  font-style: italic;  
  text-transform: uppercase;  
  letter-spacing: 1px;  
}
```

2.2.2. **Khái niệm box model: content, padding, border, margin.**

Trong CSS, mọi phần tử hiển thị trên trang đều có thể được xem như một hình hộp chữ nhật. Trình duyệt tính toán kích thước, vị trí và khoảng cách của phần tử dựa trên mô hình hộp, gọi là box model.



Hình 3.1: CSS Box Model

Box model gồm bốn lớp từ trong ra ngoài: Content, Padding, Border và Margin.

a. Content

Content là vùng chứa nội dung thực tế của phần tử, ví dụ văn bản, hình ảnh hoặc thành phần con bên trong. Khi chúng ta đặt width và height, mặc định trình duyệt hiểu đó là kích thước của vùng content. Ví dụ:

```
.box {  
  width: 300px;  
  height: 120px;  
}
```

Ở đây, 300px và 120px là kích thước của vùng nội dung, chưa tính padding, border và margin.

b. Padding

Padding là khoảng đệm nằm giữa content và border. Nó tạo không gian bên trong phần tử, giúp nội dung không dính sát vào viền. Ví dụ:

```
.box {  
  padding: 20px;  
}
```

Điều này có nghĩa là bốn phía bên trong phần tử đều được chứa 20px. Các cách viết phổ biến:

```
padding: 20px;  
padding: 10px 20px;  
padding: 10px 15px 20px 25px;
```

Ý nghĩa lần lượt:

- 20px: cả bốn phía
- 10px 20px: trên dưới 10px, trái phải 20px
- 10px 15px 20px 25px: trên, phải, dưới, trái

c. Border

Border là đường viền bao quanh padding và content. Ví dụ:

```
.box {  
  border: 2px solid #666;  
}
```

Cấu trúc khai báo border thường gồm:

- độ dày viền
- kiểu viền
- màu viền

Một số kiểu viền thông dụng:

Giá trị	Ý nghĩa
solid	nét liền
dashed	nét đứt
dotted	chấm
double	nét đôi

Bảng 3.13: Một số kiểu viền thông dụng

Border giúp làm rõ ranh giới của phần tử, đặc biệt hữu ích khi thiết kế card, form, bảng nội dung hoặc khi cần quan sát box model trong quá trình học.

d. Margin

Margin là khoảng cách bên ngoài phần tử, nằm ngoài border. Nó giúp tách phần tử này khỏi phần tử khác. Ví dụ:

```
.box {  
  margin: 24px;  
}
```

Điều này tạo khoảng cách 24px ở bốn phía bên ngoài phần tử. Điểm rất quan trọng là margin không làm tăng không gian bên trong phần tử, mà tạo khoảng trống bên ngoài. Người mới học thường nhầm margin với padding, nên cần phân biệt rõ:

Thành phần	Vị trí	Tác dụng
padding	bên trong viền	tạo khoảng đệm cho nội dung
margin	bên ngoài viền	tạo khoảng cách với phần tử khác

Bảng 3.14: Phân biệt padding và margin

e. Thuộc tính box-sizing

Để kiểm soát box model thuận tiện hơn, chúng ta thường dùng:

```
* {  
  box-sizing: border-box;  
}
```

Khi đó, width và height sẽ bao gồm cả content, padding và border. Cách này giúp tính toán bố cục dễ hơn và rất phổ biến trong phát triển giao diện hiện đại. So sánh nhanh:

Chế độ	Ý nghĩa của width
content-box	chỉ tính vùng content
border-box	tính cả content + padding + border

Bảng 3.15: So sánh content-box và border-box

2.2.3. Định dạng khối nội dung, tạo khoảng cách hợp lý giữa các phần tử.

Sau khi hiểu box model, chúng ta có thể định dạng các khối nội dung sao cho giao diện rõ ràng, dễ đọc và có khoảng cách hợp lý. Một giao diện không chỉ cần “đúng mã”, mà còn cần tổ chức không gian tốt. Trong thực tế, chất lượng thị giác của trang web phụ thuộc rất nhiều vào cách dùng padding, margin, line-height, font-size và border.

a. Nguyên tắc tạo khối nội dung dễ đọc

Khi định dạng một khối nội dung như thẻ thông tin, bài viết, hộp thông báo hoặc phần giới thiệu, chúng ta thường cần:

- chiều rộng hợp lý để dòng chữ không quá dài
- padding đủ thoáng để chữ không sát viền
- margin để khối không dính khối khác
- border hoặc màu nền để tạo ranh giới trực quan
- line-height hợp lý để đọc dễ hơn

Ví dụ:

```
.article-box {  
  max-width: 700px;  
  padding: 24px;  
  margin: 20px auto;  
  border: 1px solid #ddd;  
  background-color: #fafafa;  
}
```

Phân tích:

- max-width: 700px; giúp khối nội dung không quá rộng.
- padding: 24px; tạo khoảng đệm bên trong.
- margin: 20px auto; tạo khoảng cách trên dưới 20px và căn giữa khối.
- border tạo viền nhẹ.
- background-color giúp phân biệt vùng nội dung với nền trang.

b. Tạo khoảng cách giữa tiêu đề và đoạn văn

Ví dụ:

```
h2 {  
  margin-bottom: 12px;  
}  
p {  
  margin-bottom: 16px;  
  line-height: 1.6;  
}
```

Ở đây:

- margin-bottom của h2 tách tiêu đề khỏi đoạn ngay sau nó.
- margin-bottom của p tách các đoạn văn với nhau.
- line-height tạo khoảng cách giữa các dòng trong cùng một đoạn.

Nhờ vậy, nội dung trở nên thông thoáng và có nhịp đọc tự nhiên hơn.

c. Tạo card nội dung cơ bản

Ví dụ:

```
.card {  
  width: 300px;  
  padding: 16px;  
  border: 1px solid #ccc;  
  margin: 20px;  
  border-radius: 8px;  
}
```

Nếu bên trong có tiêu đề và mô tả:

```
.card h3 {  
  margin-top: 0;  
  margin-bottom: 10px;  
}  
.card p {  
  margin: 0;  
  line-height: 1.5;  
}
```

Cần lưu ý là nhiều thẻ HTML như h1, h2, p có sẵn margin mặc định từ trình duyệt. Vì vậy, khi định dạng card, chúng ta thường cần điều chỉnh lại margin của các phần tử con để tránh khoảng trắng thừa.

Quy trình thực hành trên VS Code

Chúng ta sẽ tạo một trang HTML để thực hành:

- định dạng văn bản
- quan sát box model
- tạo khối nội dung rõ ràng, có khoảng cách hợp lý

Bước 1. Tạo cấu trúc thư mục

box-model-demo/

```
|— index.html  
|— style.css
```

Bước 2. Tạo tệp index.html

```
<!DOCTYPE html>  
<html lang="vi">
```

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>Thực hành định dạng văn bản và box model</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="article-box">
    <h1>Giới thiệu về CSS Box Model</h1>
    <p class="intro">
      CSS giúp chúng ta kiểm soát cách hiển thị văn bản, khoảng
cách và bố cục của từng phần tử trên trang web.
    </p>
    <p>
      Khi nắm được box model, chúng ta sẽ hiểu vì sao một phần tử
chiếm diện tích lớn hơn kích thước width đã khai báo.
    </p>
    <a href="#">Đọc thêm tài liệu</a>
  </div>
  <div class="card">
    <h3>Khối nội dung minh họa</h3>
    <p>
      Khối này được định dạng bằng padding, border và margin để
tạo không gian hiển thị rõ ràng.
    </p>
  </div>
</body>
</html>
```

Bước 3. Tạo tệp style.css

```
* {
  box-sizing: border-box;
}
body {
  font-family: Arial, sans-serif;
```



```
    color: #333;
    background-color: #f5f5f5;
    margin: 0;
    padding: 20px;
}
.article-box {
    max-width: 720px;
    background-color: #ffffff;
    padding: 24px;
    margin: 20px auto;
    border: 1px solid #ddd;
}
.article-box h1 {
    font-size: 32px;
    text-align: center;
    text-decoration: underline;
    margin-bottom: 16px;
}
.article-box .intro {
    color: #1d4ed8;
    font-weight: bold;
}
.article-box p {
    font-size: 18px;
    line-height: 1.7;
    text-align: justify;
    margin-bottom: 16px;
}
.article-box a {
    color: #dc2626;
    text-decoration: none;
}
.card {
    width: 320px;
    background-color: #ffffff;
```

```
padding: 16px;
margin: 24px auto;
border: 2px solid #999;
}
.card h3 {
margin-top: 0;
margin-bottom: 12px;
text-align: center;
}
.card p {
margin: 0;
line-height: 1.5;
}
```

Bước 4. Mở trình duyệt và quan sát

Chúng ta mở index.html trong trình duyệt hoặc dùng Live Server trong VS Code để xem kết quả.

Bước 5. Phân tích kết quả

Khối .article-box

```
.article-box {
max-width: 720px;
background-color: #ffffff;
padding: 24px;
margin: 20px auto;
border: 1px solid #ddd;
}
```

Giải thích:

- max-width: 720px; giới hạn chiều rộng tối đa để dòng chữ không quá dài.
- background-color: #ffffff; làm nền trắng để nổi bật trên nền xám của trang.
- padding: 24px; tạo không gian bên trong khối.
- margin: 20px auto; tạo khoảng cách trên dưới 20px và căn giữa ngang.
- border: 1px solid #ddd; tạo đường viền nhạt.

Đoạn văn trong .article-box

```
.article-box p {
font-size: 18px;
line-height: 1.7;
```

```
text-align: justify;
margin-bottom: 16px;
}
```

Giải thích:

- font-size: 18px; tăng độ dễ đọc.
- line-height: 1.7; làm các dòng thoáng hơn.
- text-align: justify; căn đều hai mép.
- margin-bottom: 16px; tách các đoạn văn khỏi nhau.

Liên kết

```
.article-box a {
color: #dc2626;
text-decoration: none;
}
```

Giải thích:

- color đổi màu liên kết thành đỏ đậm.
- text-decoration: none; bỏ gạch chân mặc định.

Bảng sau tóm tắt phân tích lỗi thường gặp

Lỗi	Nguyên nhân kỹ thuật	Hướng xử lý
Chữ sát nhau, khó đọc	line-height quá nhỏ	Tăng line-height lên khoảng 1.5 đến 1.7
Khối nội dung chật	padding quá ít hoặc không có	Tăng padding
Các phần tử dính nhau	thiếu margin	Thêm margin-bottom hoặc margin phù hợp
Width đặt đúng nhưng phần tử vẫn tràn	chưa hiểu box model mặc định	Dùng box-sizing: border-box
Liên kết còn gạch chân dù muốn kiểu tối giản	chưa cấu hình text-decoration	Đặt text-decoration: none

Bảng 3.17; Các lỗi thường gặp và hướng xử lý

2.3. Bố cục trang với Flexbox và/hoặc Grid

Trong quá trình xây dựng giao diện web, sau khi đã nắm được cách định dạng văn bản và box model, bước tiếp theo là tổ chức các phần tử thành bố cục rõ ràng,

linh hoạt và dễ mở rộng. Đây là nhiệm vụ của các kỹ thuật layout trong CSS. Trước đây, người học thường dùng float, inline-block hoặc nhiều cách xử lý thủ công để sắp xếp phần tử theo hàng và cột. Tuy nhiên, các kỹ thuật đó có nhiều hạn chế, đặc biệt khi căn căn giữa, chia không gian linh hoạt hoặc xây dựng bố cục thích ứng theo kích thước màn hình. Flexbox và Grid ra đời để giải quyết các vấn đề này theo hướng hiện đại, trực quan và nhất quán hơn.

Flexbox phù hợp khi chúng ta muốn sắp xếp các phần tử theo một chiều chính, tức theo hàng hoặc theo cột. Grid phù hợp khi chúng ta muốn tổ chức bố cục theo hai chiều, tức vừa hàng vừa cột, với cấu trúc rõ ràng hơn. Trong giai đoạn học cơ bản, Flexbox là công cụ rất quan trọng vì nó giúp giải quyết phần lớn các bài toán bố cục phổ biến như thanh điều hướng, danh sách thẻ nội dung, bố cục hai cột, căn giữa khối nội dung hoặc chia khoảng cách đều giữa các phần tử. Grid có thể được giới thiệu sau đó như một bước mở rộng, đặc biệt hữu ích khi thiết kế trang tổng thể hoặc các khối nội dung dạng lưới.

2.3.1. Khái niệm container, item trong Flexbox; các thuộc tính chính (display, flex-direction, justify-content, align-items,...)

a. Bản chất của Flexbox

Flexbox là viết tắt của Flexible Box Layout, một mô hình bố cục CSS được thiết kế để phân phối không gian và sắp xếp các phần tử trong một container một cách linh hoạt. Khi một phần tử cha được khai báo `display: flex`, các phần tử con trực tiếp của nó trở thành các flex item. Từ thời điểm đó, trình duyệt không còn bố trí các phần tử con theo cách thông thường như block hay inline, mà áp dụng cơ chế sắp xếp của Flexbox.

Ví dụ:

```
.container {  
  display: flex;  
}
```

Nếu HTML là:

```
<div class="container">  
  <div>Phần tử 1</div>  
  <div>Phần tử 2</div>  
  <div>Phần tử 3</div>  
</div>
```

thì ba phần tử con bên trong `.container` sẽ được xếp trên cùng một hàng theo mặc định.

b. Khái niệm container và item

Trong Flexbox, chúng ta cần phân biệt rõ hai vai trò.

- **Flex container** là phần tử cha được khai báo `display: flex` hoặc `display: inline-flex`.
- **Flex item** là các phần tử con trực tiếp nằm bên trong flex container.

Ví dụ:

```
<div class="menu">
  <a href="#">Trang chủ</a>
  <a href="#">Bài học</a>
  <a href="#">Liên hệ</a>
</div>
.menu {
  display: flex;
}
```

Ở đây:

- `.menu` là container
- ba thẻ `<a>` là item

Điểm rất quan trọng là chỉ các phần tử con trực tiếp mới trở thành flex item. Nếu một item bên trong lại chứa phần tử con khác, thì các phần tử cháu đó không tự động trở thành flex item trừ khi chính item đó cũng được khai báo `display: flex`.

c. Trục chính và trục phụ

Flexbox hoạt động dựa trên hai trục:

- Main axis là trục chính
- Cross axis là trục phụ vuông góc với trục chính

Theo mặc định:

- `flex-direction: row` nghĩa là trục chính nằm ngang
- trục phụ sẽ nằm dọc

Nếu đổi thành `flex-direction: column` thì:

- trục chính nằm dọc
- trục phụ nằm ngang

Hiểu rõ hai trục này là chìa khóa để dùng đúng `justify-content` và `align-items`.

d. Thuộc tính display

Thuộc tính đầu tiên để kích hoạt Flexbox là:

```
.container {  
  display: flex;  
}
```

Giá trị flex biến phần tử thành flex container ở dạng block. Ngoài ra còn có:

```
.container {  
  display: inline-flex;  
}
```

Giá trị inline-flex làm container cư xử như phần tử inline ở bên ngoài, nhưng các phần tử con bên trong vẫn được bố trí theo Flexbox. Trong thực hành cơ bản, chúng ta chủ yếu dùng display: flex.

e. Thuộc tính flex-direction

flex-direction xác định hướng của trục chính, tức cách các item được xếp. Các giá trị cơ bản:

Giá trị	Ý nghĩa
row	Xếp theo hàng từ trái sang phải
row-reverse	Xếp theo hàng từ phải sang trái
column	Xếp theo cột từ trên xuống dưới
column-reverse	Xếp theo cột từ dưới lên trên

Bảng 3.18: Các giá trị của flex-direction

Ví dụ:

```
.container {  
  display: flex;  
  flex-direction: row;  
}
```

Các item sẽ nằm trên một hàng. Nếu đổi:

```
.container {  
  display: flex;  
  flex-direction: column;  
}
```

thì các item sẽ xếp theo cột.

f. Thuộc tính justify-content

justify-content căn chỉnh các item theo trục chính. Ví dụ với flex-direction: row, trục chính là ngang nên justify-content sẽ điều khiển vị trí các item theo chiều ngang.

Các giá trị thường dùng:

Giá trị	Ý nghĩa
flex-start	Dồn các item về đầu trục chính
flex-end	Dồn các item về cuối trục chính
center	Căn giữa theo trục chính
space-between	Item đầu ở đầu trục, item cuối ở cuối trục, khoảng giữa chia đều
space-around	Mỗi item có khoảng trống ở hai bên
space-evenly	Khoảng cách giữa các item và mép ngoài bằng nhau

Bảng 3.19: Các giá trị thường dùng của justify-content

Ví dụ:

```
.nav {  
  display: flex;  
  justify-content: space-between;  
}
```

Điều này thường được dùng cho thanh điều hướng để đẩy logo về một phía và nhóm liên kết về phía còn lại.

g. Thuộc tính align-items

align-items căn chỉnh các item theo trục phụ. Ví dụ với flex-direction: row, trục phụ là dọc, nên align-items điều khiển vị trí item theo chiều dọc.

Các giá trị thường dùng:

Giá trị	Ý nghĩa
stretch	Kéo giãn item theo trục phụ nếu không có kích thước cố định
flex-start	Dồn về đầu trục phụ
flex-end	Dồn về cuối trục phụ
center	Căn giữa theo trục phụ
baseline	Căn theo đường cơ sở của văn bản

Bảng 3.20: Các giá trị thường dùng của align-items

Ví dụ:

```
.nav {
  display: flex;
  align-items: center;
}
```

Thuộc tính này rất phổ biến để căn giữa logo và menu theo chiều dọc trong thanh điều hướng.

h. Thuộc tính gap

gap tạo khoảng cách giữa các flex item mà không cần đặt margin riêng cho từng item. Ví dụ:

```
.menu {
  display: flex;
  gap: 16px;
}
```

Ưu điểm của gap là mã ngắn gọn, rõ ràng và tránh phải xử lý margin thừa ở phần tử đầu hoặc cuối.

i. Thuộc tính flex-wrap

Mặc định, các item trong Flexbox cố gắng nằm trên một hàng hoặc một cột duy nhất. Khi không đủ chỗ, chúng có thể bị co lại quá mức hoặc tràn. flex-wrap cho phép item xuống dòng.

```
.container {
  display: flex;
  flex-wrap: wrap;
}
```

Các giá trị:

Giá trị	Ý nghĩa
nowrap	Không cho item xuống dòng
wrap	Cho phép item xuống dòng
wrap-reverse	Cho phép xuống dòng theo thứ tự đảo

Bảng 3.21: Các giá trị của flex-wrap

Thuộc tính này rất hữu ích khi làm danh sách card sản phẩm, khối tin tức hoặc bố cục cần thích nghi với nhiều độ rộng màn hình.

j. Thuộc tính flex

flex là thuộc tính rút gọn thường dùng trên item để xác định mức độ co giãn. Ví dụ:


```
.item {  
  flex: 1;  
}
```

Điều này thường làm các item chia đều không gian khả dụng trong container.
Ví dụ ba cột bằng nhau:

```
.column {  
  flex: 1;  
}
```

Nếu muốn một cột rộng hơn cột khác:

```
.left {  
  flex: 2;  
}  
.right {  
  flex: 1;  
}
```

Khi đó, cột trái sẽ chiếm không gian gấp đôi cột phải theo tỷ lệ tương đối.

k. Ví dụ tổng hợp Flexbox cơ bản

```
.container {  
  display: flex;  
  flex-direction: row;  
  justify-content: space-between;  
  align-items: center;  
  gap: 20px;  
}
```

Phân tích:

- display: flex kích hoạt Flexbox
- flex-direction: row xếp item theo hàng
- justify-content: space-between dàn đều item theo chiều ngang
- align-items: center căn giữa item theo chiều dọc
- gap: 20px tạo khoảng cách giữa các item

2.3.2. Thiết kế thanh điều hướng, bố cục hai/ba cột bằng Flexbox.

a. Thiết kế thanh điều hướng bằng Flexbox

Thanh điều hướng là ví dụ điển hình để học Flexbox vì nó thường cần sắp xếp logo, menu và nút chức năng trên cùng một hàng.

Ví dụ HTML

```
<nav class="navbar">
  <div class="logo">Học Web</div>
  <div class="menu">
    <a href="#">Trang chủ</a>
    <a href="#">Bài học</a>
    <a href="#">Bài tập</a>
    <a href="#">Liên hệ</a>
  </div>
</nav>
```

CSS

```
.navbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 16px 24px;
  background-color: #1f2937;
}
.logo {
  color: white;
  font-size: 24px;
  font-weight: bold;
}
.menu {
  display: flex;
  gap: 20px;
}
.menu a {
  color: white;
  text-decoration: none;
}
```

Phân tích kỹ thuật

- .navbar là flex container.
- justify-content: space-between đẩy logo sang trái và nhóm menu sang phải.
- align-items: center giúp các phần tử thẳng hàng theo chiều dọc.

- .menu cũng là một flex container thứ hai, dùng để xếp các liên kết ngang hàng.
- gap: 20px tạo khoảng cách đều giữa các mục menu.

Đây là ví dụ rất quan trọng vì cho thấy Flexbox có thể lồng nhau. Một container lớn điều khiển bố cục tổng thể, và một container con điều khiển bố cục của nhóm nhỏ hơn.

b. Thiết kế bố cục hai cột bằng Flexbox

Bố cục hai cột thường dùng cho trang có nội dung chính và thanh bên, hoặc phần mô tả và hình minh họa. Ví dụ HTML

```
<div class="two-column">
  <main class="content">
    <h2>Nội dung chính</h2>
    <p>Đây là khu vực chứa bài viết, tài liệu hoặc thông tin
trọng tâm.</p>
  </main>
  <aside class="sidebar">
    <h2>Thanh bên</h2>
    <p>Đây là khu vực chứa liên kết phụ, danh mục hoặc thông
báo.</p>
  </aside>
</div>
```

CSS

```
.two-column {
  display: flex;
  gap: 20px;
}
.content {
  flex: 2;
  padding: 20px;
  border: 1px solid #ccc;
}
.sidebar {
  flex: 1;
  padding: 20px;
  border: 1px solid #ccc;
}
```

Phân tích kỹ thuật

- .two-column là flex container.
- gap: 20px tạo khoảng cách giữa hai cột.
- .content { flex: 2; } làm cột nội dung chính rộng gấp đôi thanh bên.
- .sidebar { flex: 1; } làm cột phụ hẹp hơn.

Nếu muốn hai cột bằng nhau, chúng ta có thể đặt cả hai cùng flex: 1.

c. Thiết kế bố cục ba cột bằng Flexbox

Bố cục ba cột thích hợp cho khu vực hiển thị dịch vụ, tính năng, thẻ sản phẩm hoặc khối giới thiệu. Ví dụ HTML

```
<div class="three-column">
  <div class="box">Cột 1</div>
  <div class="box">Cột 2</div>
  <div class="box">Cột 3</div>
</div>
```

CSS

```
.three-column {
  display: flex;
  gap: 20px;
}
.box {
  flex: 1;
  padding: 20px;
  border: 1px solid #999;
  background-color: #f9fafb;
  text-align: center;
}
```

Phân tích kỹ thuật

- Cả ba .box có flex: 1, nên chia đều chiều rộng.
- gap: 20px tạo khoảng trống giữa các cột.
- text-align: center căn giữa nội dung trong mỗi box.

Nếu không gian màn hình nhỏ, chúng ta có thể kết hợp flex-wrap: wrap để các cột xuống dòng. Ví dụ:

```
.three-column {
  display: flex;
```

```
gap: 20px;
flex-wrap: wrap;
}
```

d. Thiết kế bố cục card xuống dòng linh hoạt

Đây là dạng thực tế hơn của bố cục nhiều cột.

HTML

```
<div class="cards">
  <div class="card">Thẻ 1</div>
  <div class="card">Thẻ 2</div>
  <div class="card">Thẻ 3</div>
  <div class="card">Thẻ 4</div>
</div>
```

CSS

```
.cards {
  display: flex;
  flex-wrap: wrap;
  gap: 20px;
}
.card {
  flex: 1 1 220px;
  padding: 20px;
  border: 1px solid #ccc;
  background-color: white;
}
```

Phân tích:

- flex-wrap: wrap cho phép xuống dòng khi thiếu chỗ.
- flex: 1 1 220px thường được hiểu là item có thể giãn, có thể co, và có kích thước cơ sở khoảng 220px.
- Khi màn hình rộng, nhiều card sẽ nằm trên cùng hàng.
- Khi màn hình hẹp, card tự xuống dòng.

Cách này rất phù hợp cho giao diện hiện đại.

2.3.3. Giới thiệu Grid layout và ứng dụng đơn giản (nếu điều kiện cho phép).

a. Bản chất của Grid layout

CSS Grid là mô hình bố cục hai chiều, cho phép chúng ta kiểm soát đồng thời hàng và cột. Nếu Flexbox mạnh trong việc sắp xếp theo một chiều, thì Grid phù hợp

hơn khi cần chia trang thành các vùng rõ ràng theo mạng lưới. Grid đặc biệt hữu ích trong các tình huống như:

- bố cục tổng thể của trang
- thư viện ảnh
- dashboard
- danh sách sản phẩm theo lưới đều

Để kích hoạt Grid:

```
.container {  
  display: grid;  
}
```

b. Khái niệm grid container và grid item

Tương tự Flexbox:

- Grid container là phần tử cha có display: grid
- Grid item là phần tử con trực tiếp bên trong

Ví dụ:

```
<div class="grid-layout">  
  <div>1</div>  
  <div>2</div>  
  <div>3</div>  
  <div>4</div>  
</div>  
.grid-layout {  
  display: grid;  
}
```

c. Thuộc tính grid-template-columns

Thuộc tính này xác định số cột và kích thước từng cột. Ví dụ:

```
.grid-layout {  
  display: grid;  
  grid-template-columns: 1fr 1fr 1fr;  
}
```

Điều này tạo ba cột có chiều rộng bằng nhau. fr là đơn vị phần tỉ lệ trong Grid. 1fr 1fr 1fr nghĩa là chia đều thành ba phần. Nếu muốn cột giữa rộng hơn:

```
grid-template-columns: 1fr 2fr 1fr;
```

d. Thuộc tính gap trong Grid

Grid cũng hỗ trợ gap để tạo khoảng cách giữa các ô.

```
.grid-layout {  
  display: grid;  
  grid-template-columns: 1fr 1fr 1fr;  
  gap: 20px;  
}
```

e. Ví dụ Grid đơn giản

HTML

```
<div class="grid-demo">  
  <div class="item">Khối 1</div>  
  <div class="item">Khối 2</div>  
  <div class="item">Khối 3</div>  
  <div class="item">Khối 4</div>  
  <div class="item">Khối 5</div>  
  <div class="item">Khối 6</div>  
</div>
```

CSS

```
.grid-demo {  
  display: grid;  
  grid-template-columns: repeat(3, 1fr);  
  gap: 20px;  
}  
.item {  
  padding: 20px;  
  border: 1px solid #999;  
  background-color: #f9fafb;  
  text-align: center;  
}
```

Phân tích:

- repeat(3, 1fr) là cách viết gọn của 1fr 1fr 1fr
- sáu item sẽ được phân bố vào ba cột
- gap tạo khoảng cách giữa các ô

f. So sánh Flexbox và Grid

Tiêu chí	Flexbox	Grid
Hướng bố cục	Một chiều	Hai chiều

Phù hợp	Thanh menu, nhóm item, card theo hàng hoặc cột	Bố cục trang, lưới nội dung
Điều khiển hàng và cột	Hạn chế	Rất mạnh
Độ dễ tiếp cận với người mới	Dễ hơn	Khó hơn một chút
Mức độ ứng dụng cơ bản	Rất cao	Bổ trợ tốt

Bảng 3.22: So sánh Flexbox và Grid

Về mặt thực hành, chúng ta nên nắm thật chắc Flexbox trước, rồi mới dùng Grid như một công cụ mở rộng.

Quy trình thực hành trên VS Code

Chúng ta sẽ xây dựng một trang đơn giản gồm:

- thanh điều hướng
- bố cục hai cột
- khối ba cột
- một vùng lưới Grid cơ bản

Bước 1. Tạo cấu trúc thư mục

layout-demo/

```
|— index.html
|— style.css
```

Bước 2. Tạo tệp index.html

```
<!DOCTYPE html>
<html lang="vi">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>Thực hành Flexbox và Grid</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <nav class="navbar">
    <div class="logo">Học Web</div>
    <div class="menu">
```



```

    <a href="#">Trang chủ</a>
    <a href="#">Bài học</a>
    <a href="#">Bài tập</a>
    <a href="#">Liên hệ</a>
  </div>
</nav>
<section class="two-column">
  <main class="content">
    <h2>Nội dung chính</h2>
    <p>Đây là khu vực chính của trang.</p>
  </main>
  <aside class="sidebar">
    <h2>Thanh bên</h2>
    <p>Đây là khu vực phụ của trang.</p>
  </aside>
</section>
<section class="three-column">
  <div class="box">Khối 1</div>
  <div class="box">Khối 2</div>
  <div class="box">Khối 3</div>
</section>
<section class="grid-demo">
  <div class="item">1</div>
  <div class="item">2</div>
  <div class="item">3</div>
  <div class="item">4</div>
  <div class="item">5</div>
  <div class="item">6</div>
</section>
</body>
</html>

```

Bước 3. Tạo tệp style.css

```

* {
  box-sizing: border-box;
}

```

```
body {
  font-family: Arial, sans-serif;
  margin: 0;
  padding: 20px;
  background-color: #f3f4f6;
  color: #333;
}
.navbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  background-color: #1f2937;
  padding: 16px 24px;
  margin-bottom: 24px;
}
.logo {
  color: white;
  font-size: 24px;
  font-weight: bold;
}
.menu {
  display: flex;
  gap: 20px;
}
.menu a {
  color: white;
  text-decoration: none;
}
.two-column {
  display: flex;
  gap: 20px;
  margin-bottom: 24px;
}
.content,
.sidebar {
```

```
padding: 20px;
background-color: white;
border: 1px solid #ddd;
}
.content {
  flex: 2;
}
.sidebar {
  flex: 1;
}
.three-column {
  display: flex;
  gap: 20px;
  margin-bottom: 24px;
}
.box {
  flex: 1;
  padding: 20px;
  background-color: white;
  border: 1px solid #ddd;
  text-align: center;
}
.grid-demo {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 20px;
}
.item {
  padding: 20px;
  background-color: white;
  border: 1px solid #ddd;
  text-align: center;
}
```

Bước 4. Mở và kiểm tra

Chúng ta mở index.html bằng trình duyệt hoặc Live Server để quan sát:

- thanh điều hướng nằm ngang
- phần nội dung và thanh bên chia theo tỷ lệ 2:1
- ba khối hiển thị đều nhau
- vùng cuối hiển thị dạng lưới ba cột

Bước 5. Thực hiện biến đổi để quan sát

Chúng ta lần lượt thử:

- đổi justify-content: space-between thành center
- đổi flex-direction: row của .three-column thành column
- thêm flex-wrap: wrap cho .three-column
- đổi grid-template-columns: repeat(3, 1fr) thành repeat(2, 1fr)

Qua đó, chúng ta sẽ quan sát trực tiếp tác động của từng thuộc tính.

Bảng sau đây tóm tắt một số lỗi thường gặp

Lỗi	Nguyên nhân kỹ thuật	Cách xử lý
justify-content không có tác dụng như mong đợi	Chưa xác định đúng trục chính	Kiểm tra lại flex-direction
align-items căn sai hướng	Nhầm trục phụ với trục chính	Xác định lại main axis và cross axis
Item không xuống dòng	Thiếu flex-wrap: wrap	Thêm flex-wrap
Cột không chia đúng tỷ lệ	Chưa dùng flex hoặc kích thước cố định xung đột	Kiểm tra lại flex, width, padding
Grid không hiện lưới	Chưa khai báo display: grid	Thêm display: grid
Khoảng cách giữa item rối	Dùng margin thủ công không đồng đều	Ưu tiên gap khi phù hợp

Bảng 3.23: Một số lỗi thường gặp khi học Flexbox và Grid

2.4. Thiết kế web responsive

Trong phát triển web hiện đại, giao diện không còn chỉ được xem trên màn hình máy tính để bàn. Người dùng có thể truy cập cùng một website bằng điện thoại, máy tính bảng, laptop, màn hình lớn hoặc thậm chí các thiết bị có tỷ lệ hiển thị đặc thù. Vì vậy, nếu giao diện chỉ được thiết kế cố định theo một kích thước duy nhất, nội dung rất dễ bị vỡ bố cục, chữ quá nhỏ, ảnh bị tràn, nút bấm khó thao tác hoặc các cột hiển thị không hợp lý. Thiết kế web responsive ra đời để giải quyết bài toán đó. Responsive web design là phương pháp xây dựng giao diện có khả năng thích nghi với nhiều kích thước màn hình khác nhau mà không cần tạo nhiều phiên bản website

tách biệt. Một website responsive phải biết co giãn, sắp xếp lại bố cục, điều chỉnh kích thước chữ, hình ảnh và khoảng cách sao cho người dùng vẫn đọc tốt, thao tác tốt và hiểu nội dung rõ ràng trên mọi thiết bị. Đây không chỉ là yêu cầu về mặt thẩm mỹ mà còn là yêu cầu kỹ thuật, trải nghiệm người dùng và khả năng bảo trì hệ thống.

2.4.1. Khái niệm responsive web design, lợi ích và nguyên tắc cơ bản.

a. Khái niệm responsive web design

Responsive web design là cách thiết kế giao diện sao cho nội dung và bố cục có thể thích nghi linh hoạt với môi trường hiển thị. Thay vì cố định chiều rộng, chiều cao và vị trí phần tử theo một khuôn duy nhất, chúng ta sử dụng các kỹ thuật CSS để phần tử co giãn, sắp xếp lại, ẩn bớt hoặc thay đổi kích thước phù hợp với từng loại màn hình.

Ví dụ, một bố cục ba cột trên máy tính có thể chuyển thành hai cột trên máy tính bảng và một cột trên điện thoại. Một thanh điều hướng ngang có thể chuyển thành menu dọc hoặc nút menu thu gọn trên màn hình nhỏ. Một hình ảnh lớn có thể tự thu nhỏ theo chiều rộng khung chứa mà không bị méo hoặc tràn.

Responsive không có nghĩa là chỉ làm “nhỏ lại”, mà là thiết kế sao cho giao diện vẫn sử dụng tốt trong từng ngữ cảnh hiển thị.

b. Lợi ích của responsive web design

Responsive web design mang lại nhiều lợi ích cả về kỹ thuật lẫn vận hành. Bảng sau đây tóm tắt một số lợi ích cơ bản

Lợi ích	Ý nghĩa kỹ thuật và thực tế
Hiển thị tốt trên nhiều thiết bị	Giao diện không bị vỡ khi thay đổi kích thước màn hình
Tăng trải nghiệm người dùng	Chữ dễ đọc, nút dễ nhấn, nội dung dễ theo dõi
Dễ bảo trì	Một mã nguồn có thể phục vụ nhiều loại thiết bị
Tiết kiệm chi phí phát triển	Không cần tạo phiên bản riêng cho desktop và mobile trong các hệ thống cơ bản
Hỗ trợ khả năng mở rộng	Thuận lợi khi thêm thành phần mới hoặc thay đổi bố cục sau này

Bảng 3.24: Lợi ích của responsive web design

Về mặt thực hành, responsive còn giúp chúng ta hình thành tư duy thiết kế linh hoạt thay vì tư duy “đóng khung theo ảnh mẫu”. Đây là một bước chuyển rất quan trọng từ làm giao diện tĩnh sang làm giao diện có khả năng thích nghi thực tế.

c. Nguyên tắc cơ bản của responsive design

Một giao diện responsive tốt thường dựa trên một số nguyên tắc cốt lõi:

Thứ nhất, bố cục phải linh hoạt: Thay vì đặt kích thước tuyệt đối cho mọi khối, chúng ta nên cho phép các phần tử co giãn tương đối theo không gian khả dụng. Flexbox và Grid là hai công cụ rất mạnh để làm điều này.

Thứ hai, nội dung phải ưu tiên khả năng đọc: Responsive không chỉ là co khung. Nếu chữ quá nhỏ, dòng quá dài hoặc khoảng cách giữa các vùng quá hẹp thì giao diện vẫn chưa đạt yêu cầu. Chúng ta cần điều chỉnh kích thước chữ, chiều cao dòng và khoảng cách để phù hợp với từng màn hình.

Thứ ba, hình ảnh và media phải mềm dẻo: Hình ảnh cần có khả năng thu nhỏ theo vùng chứa thay vì tràn ra ngoài. Nếu không xử lý tốt, chỉ một ảnh quá rộng cũng có thể làm hỏng toàn bộ giao diện trên điện thoại.

Thứ tư, giao diện cần có các điểm chuyển đổi hợp lý: Ở một số kích thước màn hình, chúng ta cần thay đổi bố cục. Chẳng hạn ba cột chuyển thành một cột, menu ngang chuyển thành menu dọc. Việc này được thực hiện bằng media queries.

d. Meta viewport và vai trò trong responsive

Để website hiển thị đúng trên thiết bị di động, tài liệu HTML cần có thẻ viewport:

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

Phân tích:

- width=device-width yêu cầu trình duyệt dùng đúng chiều rộng thực của thiết bị.
- initial-scale=1.0 thiết lập mức thu phóng ban đầu là 100%.

Nếu thiếu thẻ này, trên điện thoại trình duyệt có thể giả lập chiều rộng lớn như màn hình desktop, làm cho bố cục và kích thước chữ hiển thị sai mong đợi.

2.4.2. Sử dụng đơn vị tương đối (% , rem, vh, vw) và hình ảnh linh hoạt.

Một nền tảng rất quan trọng của responsive design là tránh phụ thuộc hoàn toàn vào đơn vị tuyệt đối như px. Thay vào đó, chúng ta sử dụng đơn vị tương đối

để phần tử thích nghi theo màn hình, theo phần tử cha hoặc theo kích thước gốc của trang.

a. Đơn vị %

Dấu % biểu diễn tỷ lệ phần trăm so với phần tử cha hoặc vùng chứa liên quan.

Ví dụ:

```
.container {  
  width: 80%;  
}
```

Nếu phần tử cha rộng 1000px thì .container rộng 800px. Nếu màn hình nhỏ hơn và vùng chứa chỉ còn 500px thì .container tự co còn 400px. % thường dùng cho:

- chiều rộng phần tử
- khoảng cách trong một số tình huống
- bố cục co giãn theo vùng chứa

Ví dụ responsive cơ bản:

```
.image-box {  
  width: 100%;  
}
```

Điều này giúp khối chiếm toàn bộ chiều rộng sẵn có của phần tử cha.

b. Đơn vị rem

rem là đơn vị tương đối dựa trên kích thước chữ gốc của phần tử gốc, thường là thẻ html. Ví dụ:

```
html {  
  font-size: 16px;  
}  
h1 {  
  font-size: 2rem;  
}
```

Kết quả:

- 1rem = 16px
- 2rem = 32px

Ưu điểm của rem là:

- dễ xây dựng hệ thống kích thước đồng bộ
- dễ thay đổi toàn bộ tỷ lệ chữ nếu chỉnh font gốc
- thuận lợi cho responsive và accessibility

Ví dụ:

```
p {  
  font-size: 1rem;  
  margin-bottom: 1rem;  
}
```

Nếu sau này chúng ta đổi kích thước gốc từ 16px thành 18px, toàn bộ phần tử dùng rem sẽ tăng theo một cách nhất quán.

c. Đơn vị vh

vh là phần trăm chiều cao của viewport, tức vùng hiển thị trình duyệt.

- 1vh tương đương 1% chiều cao màn hình
- 100vh tương đương toàn bộ chiều cao viewport

Ví dụ:

```
.hero {  
  height: 100vh;  
}
```

Khối .hero sẽ cao bằng đúng chiều cao màn hình. Đây là cách thường dùng cho màn hình mở đầu, trang giới thiệu hoặc khu vực cần chiếm trọn chiều cao hiển thị.

d. Đơn vị vw

vw là phần trăm chiều rộng của viewport.

- 1vw tương đương 1% chiều rộng màn hình
- 100vw tương đương toàn bộ chiều rộng viewport

Ví dụ:

```
.banner {  
  width: 100vw;  
}
```

Đơn vị này thường dùng trong các bài toán đặc biệt như banner toàn màn hình, kích thước chữ linh hoạt hoặc các hiệu ứng chiếm toàn bộ chiều ngang. Tuy nhiên, khi dùng vw, chúng ta cần cẩn thận vì một số trường hợp có thể gây tràn ngang nếu kết hợp không hợp lý với padding, margin hoặc thanh cuộn.

Bảng sau đây so sánh một số đơn vị thường dùng

Đơn vị	Cơ sở tính toán	Phù hợp với
--------	-----------------	-------------

px	Tuyệt đối	Kích thước cố định nhỏ, viền, chi tiết kỹ thuật
%	Theo phần tử cha hoặc vùng chứa	Chiều rộng linh hoạt
rem	Theo kích thước gốc của trang	Chữ, khoảng cách, hệ thống kích thước
vh	Theo chiều cao viewport	Khối cao theo màn hình
vw	Theo chiều rộng viewport	Khối rộng theo màn hình

Bảng 3.25: So sánh một số đơn vị thường dùng

e. Hình ảnh linh hoạt

Trong giao diện responsive, hình ảnh phải có khả năng co giãn theo vùng chứa. Một trong những quy tắc quan trọng nhất là:

```
img {
  max-width: 100%;
  height: auto;
}
```

Phân tích:

- max-width: 100% ngăn hình ảnh vượt quá chiều rộng phần tử cha
- height: auto giữ đúng tỷ lệ ảnh khi chiều rộng thay đổi

Nếu không có quy tắc này, ảnh có chiều rộng cố định lớn hơn màn hình sẽ tràn ra ngoài, làm xuất hiện thanh cuộn ngang hoặc phá vỡ bố cục. Ví dụ:

```
<div class="photo-box">
  
</div>
.photo-box {
  width: 90%;
  margin: 0 auto;
}
img {
  max-width: 100%;
  height: auto;
```

```
display: block;
}
```

Ở đây, ảnh sẽ tự co theo .photo-box.

f. Dùng max-width thay cho width trong một số trường hợp

Khi muốn một khối có thể co nhỏ trên màn hình hẹp nhưng không vượt quá một giới hạn trên màn hình rộng, chúng ta thường dùng max-width. Ví dụ:

```
.container {
  width: 100%;
  max-width: 1200px;
  margin: 0 auto;
}
```

Phân tích:

- trên màn hình nhỏ, .container có thể co về 100% chiều rộng khả dụng
- trên màn hình lớn, nó không vượt quá 1200px
- margin: 0 auto giúp căn giữa

Đây là mẫu rất phổ biến trong thiết kế responsive.

2.4.3. Giới thiệu media queries để điều chỉnh giao diện theo kích thước màn hình.

a. Khái niệm media query

Media query là cơ chế cho phép chúng ta áp dụng CSS có điều kiện dựa trên đặc điểm của thiết bị hoặc vùng hiển thị, phổ biến nhất là chiều rộng màn hình. Nói cách khác, media query giúp giao diện “ra quyết định” rằng khi màn hình nhỏ hơn hoặc lớn hơn một ngưỡng nào đó thì sẽ dùng bộ quy tắc CSS khác.

Cú pháp cơ bản:

```
@media (max-width: 768px) {
  /* CSS áp dụng khi màn hình nhỏ hơn hoặc bằng 768px */
}
```

Ví dụ này nghĩa là toàn bộ CSS bên trong sẽ chỉ có hiệu lực khi chiều rộng viewport không vượt quá 768px.

b. Hai cách tiếp cận phổ biến

Có 2 cách tiếp cận phổ biến với media query như bảng sau:

Cách tiếp cận	Ý nghĩa
max-width	Áp dụng kiểu khi màn hình nhỏ hơn hoặc bằng một ngưỡng

min-width	Áp dụng kiểu khi màn hình lớn hơn hoặc bằng một ngưỡng
-----------	--

Bảng 3.26: Hai cách tiếp cận phổ biến với media queries

max-width thường dễ hiểu hơn vì chúng ta có thể hình dung từ desktop thu nhỏ xuống tablet và mobile. Tuy nhiên trong thực tế hiện đại, nhiều dự án dùng tư duy mobile-first với min-width.

Ví dụ media query đổi bố cục hai cột thành một cột

HTML:

```
<div class="layout">
  <div class="left">Nội dung trái</div>
  <div class="right">Nội dung phải</div>
</div>
```

CSS:

```
.layout {
  display: flex;
  gap: 20px;
}
.left,
.right {
  flex: 1;
  padding: 20px;
  border: 1px solid #ccc;
}
@media (max-width: 768px) {
  .layout {
    flex-direction: column;
  }
}
```

Phân tích:

- trên màn hình rộng, .left và .right nằm cùng hàng
- khi màn hình nhỏ hơn hoặc bằng 768px, flex-direction: column làm hai khối xếp chồng dọc

Đây là một trong những ứng dụng đơn giản và quan trọng nhất của media queries.

Ví dụ media query cho thanh điều hướng

```

.navbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
}
.menu {
  display: flex;
  gap: 16px;
}
@media (max-width: 600px) {
  .navbar {
    flex-direction: column;
    align-items: flex-start;
  }
  .menu {
    flex-direction: column;
    width: 100%;
  }
}

```

Phân tích:

- trên màn hình rộng, thanh điều hướng hiển thị ngang
- trên màn hình nhỏ, toàn bộ .navbar chuyển thành bố cục dọc
- menu cũng chuyển thành danh sách dọc để dễ bấm hơn

Ví dụ media query điều chỉnh kích thước chữ

```

h1 {
  font-size: 2.5rem;
}
@media (max-width: 768px) {
  h1 {
    font-size: 2rem;
  }
}
@media (max-width: 480px) {
  h1 {
    font-size: 1.5rem;
  }
}

```

```
}  
}
```

Điều này giúp tiêu đề vẫn nổi bật trên màn hình lớn nhưng không quá to trên điện thoại.

c. Một số ngưỡng kích thước thường gặp

Các giá trị dưới đây chỉ là mốc tham khảo để thực hành

Ngưỡng	Thiết bị hoặc mục đích gần đúng
1200px	Màn hình lớn
992px	Laptop hoặc desktop nhỏ
768px	Máy tính bảng
576px hoặc 600px	Điện thoại lớn
480px	Điện thoại nhỏ

Bảng 3.28: Một số ngưỡng kích thước thường gặp

Trong giáo trình cơ bản, chúng ta không cần quá lệ thuộc vào các con số này như quy chuẩn tuyệt đối. Điều quan trọng là quan sát thực tế bố cục bắt đầu “gãy” ở đâu để đặt media query hợp lý.

Quy trình thực hành trên VS Code

Chúng ta sẽ tạo một trang đơn giản gồm:

- thanh điều hướng
- vùng giới thiệu
- bố cục ba khối nội dung
- hình ảnh linh hoạt
- media query để giao diện tự điều chỉnh

Bước 1. Tạo cấu trúc thư mục

responsive-demo/

```
|— index.html  
|— style.css
```

Bước 2. Tạo tệp index.html

```
<!DOCTYPE html>  
<html lang="vi">
```

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>Thực hành Responsive Web</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <nav class="navbar">
    <div class="logo">Học Web</div>
    <div class="menu">
      <a href="#">Trang chủ</a>
      <a href="#">Bài học</a>
      <a href="#">Tài liệu</a>
      <a href="#">Liên hệ</a>
    </div>
  </nav>
  <section class="hero">
    <h1>Thiết kế web responsive</h1>
    <p>
      Giao diện cần thích nghi với nhiều kích thước màn hình để
      người dùng đọc và thao tác thuận tiện.
    </p>
  </section>
  <section class="cards">
    <div class="card">Nội dung 1</div>
    <div class="card">Nội dung 2</div>
    <div class="card">Nội dung 3</div>
  </section>
  <section class="image-section">
    
  </section>
</body>
</html>
```

Bước 3. Tạo tệp style.css

```
* {  
  box-sizing: border-box;  
}  
html {  
  font-size: 16px;  
}  
body {  
  margin: 0;  
  font-family: Arial, sans-serif;  
  color: #333;  
  background-color: #f5f5f5;  
}  
.navbar {  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
  padding: 1rem 2rem;  
  background-color: #1f2937;  
}  
.logo {  
  color: white;  
  font-size: 1.5rem;  
  font-weight: bold;  
}  
.menu {  
  display: flex;  
  gap: 1rem;  
}  
.menu a {  
  color: white;  
  text-decoration: none;  
}  
.hero {  
  width: 90%;
```

```
    max-width: 1200px;
    margin: 2rem auto;
    padding: 2rem;
    background-color: white;
}
.hero h1 {
    font-size: 2.5rem;
    margin-top: 0;
}
.hero p {
    font-size: 1rem;
    line-height: 1.7;
}
.cards {
    width: 90%;
    max-width: 1200px;
    margin: 0 auto 2rem auto;
    display: flex;
    gap: 1.25rem;
}
.card {
    flex: 1;
    padding: 1.5rem;
    background-color: white;
    border: 1px solid #ddd;
    text-align: center;
}
.image-section {
    width: 90%;
    max-width: 1200px;
    margin: 0 auto 2rem auto;
}
.image-section img {
    max-width: 100%;
    height: auto;
}
```



```
display: block;
}
@media (max-width: 768px) {
  .navbar {
    flex-direction: column;
    align-items: flex-start;
    gap: 1rem;
  }
  .menu {
    flex-direction: column;
  }
  .cards {
    flex-direction: column;
  }
  .hero h1 {
    font-size: 2rem;
  }
}
@media (max-width: 480px) {
  html {
    font-size: 15px;
  }
  .navbar {
    padding: 1rem;
  }
  .hero {
    width: 95%;
    padding: 1rem;
  }
  .hero h1 {
    font-size: 1.6rem;
  }
}
```

Bước 4. Mở trong trình duyệt

Chúng ta mở index.html bằng trình duyệt hoặc Live Server.

Bước 5. Kiểm tra responsive

Trong trình duyệt, mở công cụ dành cho nhà phát triển và chuyển sang chế độ mô phỏng thiết bị. Sau đó thu nhỏ dần chiều rộng màn hình để quan sát:

- menu ngang chuyển thành menu dọc
- ba thẻ nội dung chuyển từ hàng ngang thành xếp dọc
- tiêu đề giảm kích thước
- khối nội dung co lại theo chiều rộng màn hình
- ảnh tự co theo vùng chứa mà không bị méo

Một số lỗi thường gặp khi làm responsive

Lỗi	Nguyên nhân kỹ thuật	Cách xử lý
Giao diện trên điện thoại quá nhỏ	Thiếu thẻ viewport	Thêm thẻ meta viewport vào phần head
Ảnh tràn khỏi khối chứa	Ảnh dùng kích thước cố định, không co giãn	Dùng max-width: 100% và height: auto
Bố cục nhiều cột quá chật trên mobile	Không có media query đổi hướng bố cục	Dùng media query và chuyển về một cột
Chữ trên điện thoại quá to hoặc quá nhỏ	Dùng kích thước không phù hợp, không điều chỉnh theo màn hình	Dùng rem và media query
Xuất hiện thanh cuộn ngang	Phần tử quá rộng, dùng width hoặc vw không kiểm soát	Kiểm tra width, padding, margin, ảnh và container

Bảng 3.29: Một số lỗi thường gặp khi làm responsive

2.5. Thực hành tổng hợp bài 2

Phần thực hành tổng hợp của Bài 2 có vai trò kết nối toàn bộ kiến thức CSS đã học trước đó thành một sản phẩm giao diện hoàn chỉnh. Nếu ở các mục trước, chúng ta lần lượt học từng nhóm kỹ thuật như selector, định dạng văn bản, box model, Flexbox, Grid và responsive design, thì ở mục này, chúng ta đưa các kỹ thuật đó vào một bài tập gần với tình huống thực tế hơn. Mục tiêu không chỉ là viết đúng mã CSS,

mà còn là biết tổ chức giao diện thành các khu vực rõ ràng, đảm bảo tính thẩm mỹ, khả năng đọc, bố cục hợp lý và khả năng thích nghi trên nhiều loại màn hình.

Bài thực hành này giả định rằng ở Bài 1 chúng ta đã xây dựng được cấu trúc HTML cơ bản cho một website nhỏ, chẳng hạn website giới thiệu cá nhân, lớp học, nhóm dự án hoặc cửa hàng nhỏ. Ở Bài 2, nhiệm vụ của chúng ta là dùng CSS để biến bộ khung HTML đó thành một giao diện hoàn chỉnh, hiện đại, dễ đọc và có khả năng responsive.

2.5.1. Áp dụng CSS để tạo giao diện hoàn chỉnh cho website đã xây dựng ở Bài 1.

a. Mục tiêu của bài thực hành

Trong phần này, chúng ta không tạo mới toàn bộ website từ đầu, mà kế thừa cấu trúc HTML đã xây dựng ở Bài 1. Công việc trọng tâm là:

- chuẩn hóa phần hiển thị chung của toàn trang
- định dạng các vùng nội dung theo phong cách thống nhất
- tạo hệ thống khoảng cách, màu sắc, cỡ chữ, khối hiển thị
- tổ chức bố cục theo hướng hiện đại, dễ đọc và dễ mở rộng

Một website hoàn chỉnh về mặt giao diện thường cần có ít nhất các khu vực như bảng sau:

Khu vực	Vai trò
Header hoặc menu	điều hướng đến các trang hoặc các phần nội dung
Banner hoặc hero section	giới thiệu nổi bật nội dung chính
Khu vực nội dung chính	trình bày bài viết, dịch vụ, sản phẩm hoặc thông tin
Footer	thông tin liên hệ, bản quyền, liên kết phụ

Bảng 3.30: Các khu vực chính của website hoàn chỉnh

Khi triển khai CSS, chúng ta cần thống nhất phong cách giữa các khu vực này để giao diện không rời rạc.

b. Quy trình áp dụng CSS cho website đã có HTML

Để tránh viết CSS rời rạc, chúng ta nên thực hiện theo trình tự sau.

Bước 1. Tạo tệp CSS và liên kết với HTML

Ví dụ cấu trúc thư mục:

bai2-practice/

└─ index.html

└─ style.css

Trong index.html, liên kết CSS:

```
<link rel="stylesheet" href="style.css">
```

Nếu thiếu bước này, HTML vẫn mở được nhưng mọi CSS đều không có tác dụng.

Bước 2. Thiết lập kiểu chung cho toàn trang

Chúng ta thường bắt đầu bằng một số quy tắc nền để đồng bộ cách trình duyệt hiển thị.

```
* {  
  box-sizing: border-box;  
}  
body {  
  margin: 0;  
  font-family: Arial, sans-serif;  
  color: #333;  
  background-color: #f5f5f5;  
  line-height: 1.6;  
}
```

Giải thích:

- box-sizing: border-box; giúp tính kích thước phần tử thuận tiện hơn.
- margin: 0; loại bỏ khoảng trắng mặc định quanh body.
- font-family đặt phông chữ chung.
- color đặt màu chữ mặc định.
- background-color tạo nền nhẹ cho toàn trang.
- line-height cải thiện khả năng đọc.

Bước 3. Tạo lớp khung nội dung dùng chung

Trong website thực tế, chúng ta thường không để nội dung kéo dài toàn bộ chiều ngang màn hình lớn. Vì vậy cần một lớp khung giới hạn chiều rộng.

```
.container {  
  width: 90%;  
  max-width: 1200px;
```

```
margin: 0 auto;
}
```

Lớp .container có thể dùng lặp lại trong nhiều khu vực như header, banner, content, footer.

Bước 4. Phân vùng và định dạng từng khu vực

Lúc này chúng ta mới đi vào từng vùng như menu, banner, nội dung, footer. Cách làm đúng kỹ thuật là xác định rõ vai trò của từng phần tử trong HTML rồi viết CSS theo khu vực, không nên viết lẫn lộn.

c. Mẫu HTML tổng hợp cho website thực hành

Dưới đây là một cấu trúc mẫu để chúng ta thực hành hoàn chỉnh Bài 2.

```
<!DOCTYPE html>
<html lang="vi">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>Website thực hành Bài 2</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <header class="site-header">
    <div class="container navbar">
      <div class="logo">Học Web</div>
      <nav class="menu">
        <a href="#">Trang chủ</a>
        <a href="#">Giới thiệu</a>
        <a href="#">Dịch vụ</a>
        <a href="#">Liên hệ</a>
      </nav>
    </div>
  </header>
  <section class="hero">
    <div class="container">
      <h1>Thiết kế giao diện web hiện đại với CSS</h1>
      <p>
```

Chúng ta áp dụng CSS để biến cấu trúc HTML cơ bản thành một website rõ ràng, dễ đọc và thích nghi tốt trên nhiều thiết bị.

```
</p>
<a href="#" class="btn">Tìm hiểu thêm</a>
</div>
</section>
<main class="container main-content">
  <section class="cards">
    <article class="card">
      <h2>Bài học 1</h2>
      <p>Ôn lại cấu trúc HTML và tổ chức nội dung website.</p>
    </article>
    <article class="card">
      <h2>Bài học 2</h2>
      <p>Áp dụng CSS để định dạng giao diện và bố cục.</p>
    </article>
    <article class="card">
      <h2>Bài học 3</h2>
      <p>Kiểm tra responsive và điều chỉnh hiển thị trên
mobile.</p>
    </article>
  </section>
  <section class="info-section">
    <h2>Giới thiệu khóa học</h2>
    <p>
      Nội dung giúp học sinh, sinh viên nắm được cách định dạng
      văn bản, hộp, bố cục và responsive để xây dựng website hoàn
      chỉnh.
    </p>
  </section>
</main>
<footer class="site-footer">
  <div class="container">
    <p>© 2026 Học Web Cơ Bản. All rights reserved.</p>
```

```
    </div>
  </footer>
</body>
</html>
```

d. Mẫu CSS hoàn chỉnh cho giao diện tổng hợp

```
* {
  box-sizing: border-box;
}
body {
  margin: 0;
  font-family: Arial, sans-serif;
  color: #333;
  background-color: #f5f5f5;
  line-height: 1.6;
}
.container {
  width: 90%;
  max-width: 1200px;
  margin: 0 auto;
}
.site-header {
  background-color: #111827;
}
.navbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 16px 0;
}
.logo {
  color: white;
  font-size: 24px;
  font-weight: bold;
}
.menu {
```

```
    display: flex;
    gap: 20px;
}
.menu a {
    color: white;
    text-decoration: none;
    font-weight: 500;
}
.hero {
    background: linear-gradient(135deg, #1d4ed8, #2563eb);
    color: white;
    padding: 60px 0;
    text-align: center;
}
.hero h1 {
    margin-top: 0;
    font-size: 40px;
}
.hero p {
    max-width: 750px;
    margin: 0 auto 24px auto;
    font-size: 18px;
}
.btn {
    display: inline-block;
    padding: 12px 24px;
    background-color: white;
    color: #1d4ed8;
    text-decoration: none;
    border-radius: 6px;
    font-weight: bold;
}
.main-content {
    padding: 40px 0;
}
```



```
.cards {
  display: flex;
  gap: 20px;
  margin-bottom: 32px;
}

.card {
  flex: 1;
  background-color: white;
  padding: 24px;
  border: 1px solid #ddd;
  border-radius: 8px;
}

.card h2 {
  margin-top: 0;
  margin-bottom: 12px;
}

.info-section {
  background-color: white;
  padding: 24px;
  border: 1px solid #ddd;
  border-radius: 8px;
}

.info-section h2 {
  margin-top: 0;
}

.site-footer {
  background-color: #111827;
  color: white;
  text-align: center;
  padding: 20px 0;
}
```

Đây là một bộ CSS tương đối đầy đủ cho website nhỏ. Nó kết hợp:

- định dạng tổng thể
- Flexbox cho menu và card
- box model cho khoảng cách và khung hiển thị

- màu sắc hiện đại, rõ tương phản
- tổ chức theo từng khu vực rõ ràng

2.5.2. Thiết kế menu, banner, khối nội dung, footer theo phong cách hiện đại.

a. Đặc điểm của phong cách giao diện hiện đại

Khi nói “phong cách hiện đại” trong bối cảnh giáo trình cơ bản, chúng ta không nhắm đến hiệu ứng phức tạp, mà tập trung vào một số đặc điểm dễ nhận biết:

- bố cục rõ ràng, thoáng
- khoảng cách hợp lý
- màu sắc ít nhưng nhất quán
- chữ dễ đọc
- thành phần giao diện có phân cấp rõ
- không dùng quá nhiều viền nặng hoặc nền rối
- nút bấm, card, menu có trạng thái và hình thức sạch

Nói cách khác, hiện đại trong giai đoạn đầu là sự rõ ràng, tối giản, dễ sử dụng và có tổ chức.

b. Thiết kế menu

Menu là thành phần đầu tiên người dùng nhìn thấy. Một menu hiện đại nên có:

- vị trí rõ ràng
- liên kết dễ đọc
- khoảng cách đều
- màu tương phản tốt với nền
- bố cục cân đối giữa logo và các mục điều hướng

Ví dụ CSS cho menu:

```
.navbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 16px 0;
}
.menu {
  display: flex;
  gap: 20px;
}
.menu a {
```

```
color: white;
text-decoration: none;
font-weight: 500;
}
```

Giải thích:

- display: flex đặt logo và menu trên cùng một hàng.
- justify-content: space-between đẩy hai nhóm về hai phía.
- gap tạo khoảng cách đồng đều giữa các liên kết.
- text-decoration: none loại bỏ gạch chân mặc định để giao diện gọn hơn.

Nếu muốn tăng cảm giác hiện đại hơn, chúng ta có thể thêm hiệu ứng hover:

```
.menu a:hover {
  color: #93c5fd;
}
```

c. Thiết kế banner hoặc hero section

Banner là vùng nổi bật để giới thiệu nội dung cốt lõi của trang. Về mặt kỹ thuật, banner hiện đại thường có:

- nền màu hoặc nền chuyển sắc
- tiêu đề lớn, rõ
- đoạn mô tả ngắn
- nút hành động
- khoảng đệm lớn để tạo cảm giác nổi bật

Ví dụ:

```
.hero {
  background: linear-gradient(135deg, #1d4ed8, #2563eb);
  color: white;
  padding: 60px 0;
  text-align: center;
}
```

Phân tích:

- linear-gradient(...) tạo nền chuyển sắc hiện đại hơn màu đơn.
- color: white tạo độ tương phản tốt.
- padding: 60px 0 giúp banner có không gian lớn và thoáng.
- text-align: center giúp nội dung tập trung ở trung tâm.

d. Thiết kế khối nội dung

Khối nội dung hiện đại thường dùng card hoặc section có nền trắng, viền mảnh, bo góc nhẹ và khoảng cách hợp lý. Ví dụ:

```
.card {  
  flex: 1;  
  background-color: white;  
  padding: 24px;  
  border: 1px solid #ddd;  
  border-radius: 8px;  
}
```

Các điểm kỹ thuật:

- background-color: white làm card nổi trên nền xám của trang.
- padding tạo khoảng thở cho nội dung.
- border giúp xác định ranh giới card.
- border-radius làm card mềm hơn về mặt thị giác.

Chúng ta cũng có thể dùng Flexbox để các card nằm ngang trên màn hình rộng:

```
.cards {  
  display: flex;  
  gap: 20px;  
}
```

e. Thiết kế footer

Footer là khu vực kết thúc trang, thường chứa thông tin bản quyền, liên hệ hoặc liên kết bổ sung. Một footer cơ bản nhưng hiện đại nên có:

- nền tách biệt với nội dung chính
- chữ rõ ràng, không quá lớn
- khoảng cách trên dưới vừa phải
- bố cục gọn

Ví dụ:

```
.site-footer {  
  background-color: #111827;  
  color: white;  
  text-align: center;
```

```
padding: 20px 0;
}
```

Footer không nên quá phức tạp trong giai đoạn đầu, vì mục tiêu chính là hoàn thiện cấu trúc giao diện tổng thể.

Bảng tổng hợp vai trò các khu vực giao diện

Thành phần	Mục đích	Thuộc tính CSS thường dùng
Menu	điều hướng	display: flex, gap, align-items
Banner	giới thiệu nổi bật	background, padding, text-align
Card nội dung	nhóm thông tin	padding, border, border-radius, flex
Footer	kết thúc trang, thông tin phụ	background-color, padding, text-align

Bảng 3.31: Vai trò các thành phần giao diện theo phong cách hiện đại

2.5.3. Kiểm tra hiển thị website trên nhiều độ rộng màn hình và điều chỉnh cho phù hợp

a. Mục tiêu của bước kiểm tra responsive

Sau khi hoàn thiện giao diện desktop, chúng ta chưa thể xem bài thực hành là hoàn chỉnh nếu chưa kiểm tra khả năng hiển thị trên nhiều độ rộng màn hình. Một giao diện có thể nhìn tốt trên laptop nhưng lỗi trên điện thoại do:

- menu quá dài
- các card quá hẹp
- tiêu đề quá lớn
- khoảng cách không còn hợp lý
- nội dung bị tràn ngang

Vì vậy, bước kiểm tra responsive là bắt buộc trong quy trình thực hành.

b. Cách kiểm tra trên VS Code và trình duyệt

Sau khi mở website bằng trình duyệt, chúng ta thực hiện:

1. nhấn chuột phải trên trang và chọn **Inspect**
2. mở chế độ mô phỏng thiết bị
3. thử các độ rộng phổ biến như 1200px, 992px, 768px, 600px, 480px
4. quan sát menu, banner, card, đoạn văn và footer

Những nội dung cần kiểm tra theo bảng sau:

Khu vực	Cần quan sát
Menu	có bị dồn chặt, xuống dòng xấu hay không
Banner	tiêu đề có quá lớn trên mobile không
Card	có còn đọc được khi màn hình hẹp không
Nội dung văn bản	có bị dòng quá dài hoặc quá chặt không
Toàn trang	có xuất hiện thanh cuộn ngang không

Bảng 3.32: Nội dung cần kiểm tra khi thử responsive

c. Bổ sung media queries để điều chỉnh

Dưới đây là một bộ media query phù hợp cho bài thực hành cơ bản

```
@media (max-width: 768px) {
  .navbar {
    flex-direction: column;
    align-items: flex-start;
    gap: 12px;
  }
  .menu {
    flex-direction: column;
    gap: 10px;
  }
  .hero h1 {
    font-size: 32px;
  }
  .cards {
    flex-direction: column;
  }
}
```

Phân tích:

- menu tổng thể chuyển từ ngang sang dọc
- nhóm liên kết trong menu cũng xếp dọc
- tiêu đề banner giảm kích thước
- các card chuyển thành một cột để dễ đọc hơn

Nếu cần tối ưu hơn cho điện thoại nhỏ:

```
@media (max-width: 480px) {  
  .hero {  
    padding: 40px 0;  
  }  
  .hero h1 {  
    font-size: 26px;  
  }  
  .hero p {  
    font-size: 16px;  
  }  
  .btn {  
    padding: 10px 18px;  
  }  
  .card,  
  .info-section {  
    padding: 18px;  
  }  
}
```

Khi đó, toàn bộ giao diện sẽ gọn hơn trên màn hình nhỏ mà vẫn giữ được tính rõ ràng.

Quy trình thực hành trên VS Code

Bước 1. Chuẩn bị dự án

Tạo thư mục:

tong-hop-bai-2/

├── index.html

└── style.css

Dùng mẫu HTML và CSS ở phần trên.

Bước 2. Chạy giao diện trong trình duyệt

Mở index.html bằng Live Server hoặc trình duyệt thông thường.

Bước 3. Kiểm tra giao diện desktop

Xác nhận:

- menu nằm đúng vị trí
- banner có khoảng cách thoáng

- ba card hiển thị đều
- footer nằm cuối trang

Bước 4. Kiểm tra giao diện tablet và mobile

Thu nhỏ cửa sổ hoặc dùng chế độ mô phỏng thiết bị. Quan sát:

- menu có bị sát nhau không
- card có quá chật không
- tiêu đề banner có chiếm nhiều dòng không
- nội dung có bị tràn ngang không

Bước 5. Thêm media query

Chèn phần sau vào cuối style.css:

```
@media (max-width: 768px) {  
  .navbar {  
    flex-direction: column;  
    align-items: flex-start;  
    gap: 12px;  
  }  
  .menu {  
    flex-direction: column;  
    gap: 10px;  
  }  
  .cards {  
    flex-direction: column;  
  }  
  .hero h1 {  
    font-size: 32px;  
  }  
}  
  
@media (max-width: 480px) {  
  .hero {  
    padding: 40px 0;  
  }  
  .hero h1 {  
    font-size: 26px;  
  }  
}
```



```
.hero p {
  font-size: 16px;
}
}
```

Bước 6. Kiểm tra lại sau khi điều chỉnh

Chúng ta lặp lại việc kiểm tra trên các độ rộng khác nhau để xác nhận giao diện đã thích nghi tốt hơn.

Một số lỗi thường gặp trong bài thực hành tổng hợp:

Lỗi	Nguyên nhân kỹ thuật	Cách xử lý
Menu bị lệch hoặc chồng nhau	thiếu display: flex hoặc khoảng cách chưa hợp lý	kiểm tra .navbar, .menu, gap
Banner nhìn chật	padding quá nhỏ hoặc tiêu đề quá lớn	tăng padding, giảm font-size trên mobile
Card không đều nhau	thiếu flex: 1 hoặc có nội dung quá chênh lệch	dùng flex: 1, kiểm tra nội dung
Footer không nổi bật	màu nền và màu chữ thiếu tương phản	tăng tương phản màu
Website có thanh cuộn ngang	phần tử quá rộng hoặc ảnh không linh hoạt	kiểm tra width, ảnh, max-width: 100%
Giao diện desktop đẹp nhưng mobile vỡ	chưa bổ sung media query	thêm media queries phù hợp

Bảng 3.33: Một số lỗi thường gặp trong bài thực hành tổng hợp

Tổng kết

Bài 2 trang bị cho người học kiến thức và kỹ năng cốt lõi để định dạng giao diện và tổ chức bố cục trang web bằng CSS. Từ nền tảng HTML đã xây dựng ở Bài 1, người học biết sử dụng selector, thuộc tính và giá trị để điều khiển cách hiển thị của từng phần tử, đồng thời hiểu được ba cách áp dụng CSS và lựa chọn hình thức phù hợp trong quá trình phát triển website.

Trong quá trình học, người học được rèn luyện khả năng định dạng văn bản, màu sắc, kích thước, khoảng cách và đường viền thông qua box model; từ đó tạo ra các khối nội dung rõ ràng, cân đối và dễ quan sát. Việc tiếp cận Flexbox hoặc Grid giúp người học không chỉ biết chỉnh sửa hình thức bề ngoài mà còn biết sắp xếp cấu trúc giao diện theo tư duy bố cục, đáp ứng yêu cầu thẩm mỹ và khả năng sử dụng.

Một kết quả quan trọng của bài học là người học nhận thức được ý nghĩa của responsive web design trong bối cảnh website phải hiển thị tốt trên nhiều loại thiết bị. Sau khi hoàn thành bài, người học cần có khả năng liên kết CSS với HTML, định dạng các thành phần chính của trang, xây dựng bố cục cơ bản bằng Flexbox hoặc Grid, sử dụng đơn vị tương đối và thiết lập media queries để điều chỉnh giao diện. Đây là cơ sở trực tiếp để chuyển sang Bài 3, nơi JavaScript sẽ được bổ sung nhằm tạo ra các tương tác và hành vi động trên giao diện đã được định dạng bằng HTML và CSS.

Bài tập và đánh giá

A. Câu hỏi lý thuyết (5 điểm)

Câu 1 (1 điểm) – Selector, thuộc tính và cách áp dụng CSS

Trình bày khái niệm selector, thuộc tính và giá trị trong CSS. Phân biệt ba cách áp dụng CSS gồm nội tuyến, nội bộ và tách riêng tệp CSS; nêu ưu điểm cơ bản của việc tách riêng tệp CSS trong một website có nhiều trang.

Câu 2 (1 điểm) – Box model và định dạng phần tử

Giải thích các thành phần của box model gồm content, padding, border, margin. Cho biết padding và margin khác nhau ở điểm nào và việc hiểu đúng box model có ý nghĩa gì khi thiết kế bố cục trang web.

Câu 3 (1 điểm) – Bố cục với Flexbox hoặc Grid

Trình bày vai trò của Flexbox hoặc Grid trong việc xây dựng bố cục trang web. Nêu một ví dụ cụ thể về việc dùng Flexbox để tạo thanh điều hướng hoặc bố cục hai cột.

Câu 4 (1 điểm) – Responsive web design

Giải thích khái niệm responsive web design. Vì sao khi thiết kế web hiện nay cần quan tâm đến nhiều kích thước màn hình khác nhau? Nêu vai trò của đơn vị tương đối và media queries.

Câu 5 (1 điểm) – Liên hệ giữa HTML và CSS trong thiết kế giao diện

Phân tích mối quan hệ giữa HTML và CSS trong quá trình xây dựng website. Vì sao một trang HTML đúng cấu trúc vẫn cần CSS để trở nên dễ sử dụng và có tính thẩm mỹ?

B. Câu hỏi thực hành (5 điểm)

Thực hành 1 (2,5 điểm)

Yêu cầu:

Sử dụng bộ tệp HTML đã xây dựng ở Bài 1 hoặc tạo mới một trang web giới thiệu đơn giản gồm các phần: tiêu đề, menu điều hướng, khối nội dung chính và chân trang. Viết tệp CSS riêng để định dạng phong chữ, màu sắc, khoảng cách, đường viền và nền cho các khu vực chính. Trong đó cần thể hiện rõ việc sử dụng selector theo thẻ, class hoặc id một cách hợp lý.

Nội dung sản phẩm cần có:

- Liên kết đúng tệp CSS từ trang HTML.
- Định dạng ít nhất các thành phần: tiêu đề trang, menu, đoạn văn và một khối nội dung hoặc hình ảnh.
- Áp dụng box model để tạo khoảng cách rõ ràng giữa các phần tử.
- Tổ chức giao diện cân đối, dễ đọc và nhất quán màu sắc.

Thực hành 2 (2,5 điểm)

Yêu cầu:

Phát triển tiếp sản phẩm ở Thực hành 1 theo hướng responsive. Người học sử dụng Flexbox hoặc Grid để xây dựng bố cục hai cột hoặc ba khu vực chính; sau đó bổ sung media queries để giao diện tự điều chỉnh khi màn hình thu hẹp xuống kích thước máy tính bảng hoặc điện thoại.

Nội dung sản phẩm cần có:

- Sử dụng Flexbox hoặc Grid để tổ chức bố cục.
- Bố cục hiển thị hợp lý ở màn hình rộng và tự sắp xếp lại khi màn hình nhỏ.
- Dùng ít nhất một đơn vị tương đối như %, rem, vw hoặc vh.
- Có tối thiểu một media query để điều chỉnh menu, cột hoặc kích thước phần tử.

C. Bảng đánh giá (thang 10 điểm)

C.1. Phân bố điểm

Nhóm	Nội dung	Điểm
Lý thuyết	Câu 1 - Câu 5	5,0

Thực hành	Thực hành 1	2,5
Thực hành	Thực hành 2	2,5
Tổng		10,0

C.2. Chấm câu lý thuyết (mỗi câu tối đa 1 điểm)

Mức	Điểm	Tiêu chí	Gợi ý lỗi thường gặp
Đạt tối đa	1,0	Trả lời đúng trọng tâm, đầy đủ ý chính; dùng thuật ngữ CSS chính xác; có ví dụ phù hợp liên quan selector, box model, Flexbox/Grid và responsive web design.	Thiếu ví dụ; nhầm giữa padding và margin; giải thích mơ hồ về selector; chưa làm rõ vai trò của media queries.
Đạt một phần	0,5	Nêu được ý chính nhưng thiếu chi tiết quan trọng; ví dụ chưa rõ hoặc chưa liên hệ đúng với thiết kế giao diện web.	Mô tả chung chung; chưa phân biệt rõ các cách áp dụng CSS; chưa chỉ ra được ý nghĩa thực tế của responsive.
Chưa đạt	0,0	Sai trọng tâm hoặc thiếu phần lớn nội dung.	Nhầm lẫn cơ bản, không trả lời được câu hỏi.

C.3. Chấm Thực hành 1 (tối đa 2,5 điểm)

Tiêu chí	Điểm tối đa	Đạt tối đa	Đạt một phần	Chưa đạt	Minh chứng cần có
Liên kết và tổ chức tệp CSS	0,5	Liên kết đúng tệp CSS; cấu trúc tệp rõ ràng.	Có liên kết nhưng sai tên tệp hoặc tổ chức chưa gọn.	Không liên kết được CSS hoặc tệp lỗi.	Tệp HTML, tệp CSS, ảnh chụp giao diện hiển thị trên trình duyệt hoặc video
Định dạng các thành phần cơ bản	0,75	Tiêu đề, menu, đoạn văn, khối nội	Mới định dạng một phần; giao	Hầu như chưa có định dạng CSS.	

		dung được định dạng rõ ràng.	diện còn rời rạc.		thao tác ngắn.
Vận dụng box model	0,75	Sử dụng hợp lý padding, margin, border để tạo khoảng cách rõ.	Có áp dụng nhưng chưa đều hoặc còn chật chội.	Chưa thể hiện box model rõ ràng.	
Tính thẩm mỹ và nhất quán	0,5	Màu sắc, cỡ chữ, khoảng cách tương đối hài hòa.	Giao diện còn thiếu cân đối nhưng vẫn đọc được.	Giao diện rời, khó đọc, thiếu nhất quán.	

C.4. Chấm Thực hành 2 (tối đa 2,5 điểm)

Tiêu chí	Điểm tối đa	Đạt tối đa	Đạt một phần	Chưa đạt	Minh chứng cần có
Tổ chức bố cục bằng Flexbox/Grid	0,75	Bố cục rõ ràng, dùng đúng thuộc tính của Flexbox hoặc Grid.	Có dùng nhưng bố cục chưa ổn định hoặc chưa hợp lý.	Không thể hiện rõ việc dùng Flexbox/Grid.	Bộ tệp hoàn chỉnh, ảnh chụp ở ít nhất hai kích thước màn hình hoặc video thể hiện giao diện thay đổi khi thu/phóng cửa sổ
Khả năng thích ứng màn hình	0,75	Giao diện thay đổi phù hợp ở màn hình nhỏ.	Có thay đổi nhưng còn lỗi căn chỉnh hoặc chồng lấn.	Không thích ứng hoặc hiển thị lỗi.	
Sử dụng đơn vị tương đối và media query	0,5	Có dùng đúng đơn vị tương đối	Có dùng nhưng chưa rõ	Không có hoặc sai hoàn toàn.	

		và ít nhất một media query.	tác dụng hoặc viết chưa chính xác.		trình duyệt.
Mức độ hoàn thiện sản phẩm	0,5	Trang web hoạt động ổn định, dễ quan sát và dễ đọc.	Sản phẩm cơ bản dùng được nhưng còn nhiều điểm chưa gọn.	Sản phẩm thiếu, lỗi nhiều hoặc khó sử dụng.	